What Is SQL, everything you need to know:

Agenda

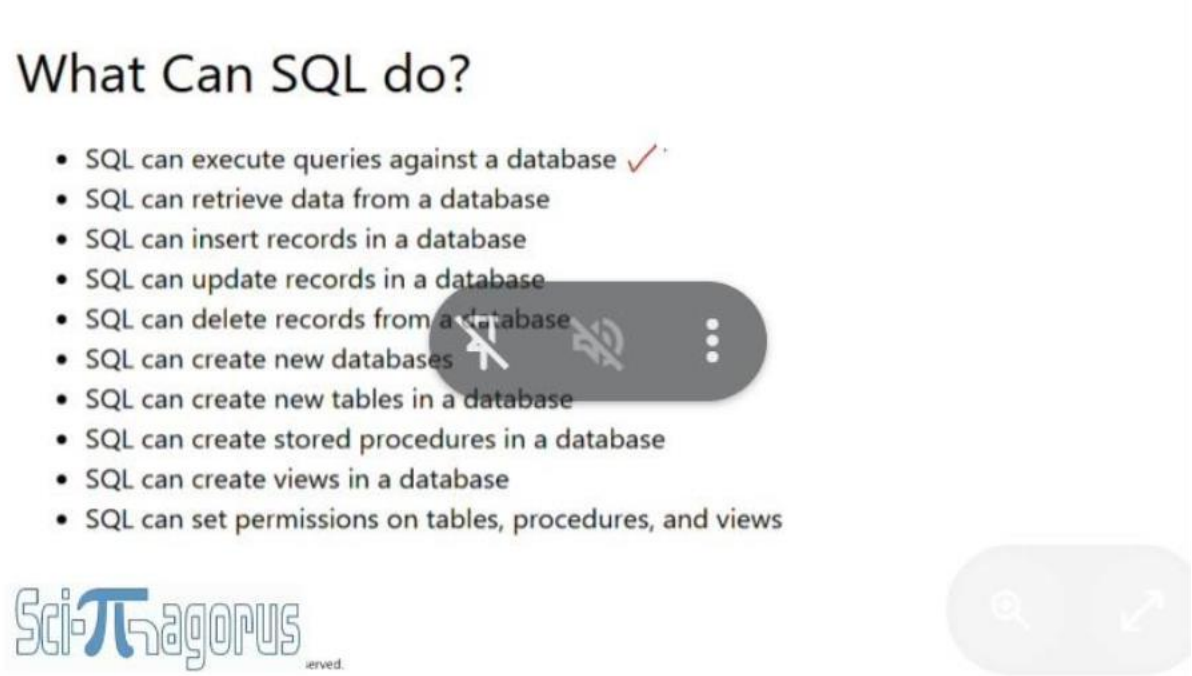
- Introduction to Transact-SQL
- Understanding Tables, Rows, Columns
- Understanding Data Types
- Understanding Operators
- Writing Basic SELECT Statement
- Sorting data using ORDER BY
- Limiting Columns and Rows
- Using Simple Operators (=, >, <, LIKE, IN)
- Using Constraints

© Copyright Microsoft Corporation. All rights reserved.

What Can SQL do?

- SQL can execute queries against a database
- SQL can retrieve data from a database
- SQL can insert records in a database
- SQL can update records in a database
- SQL can delete records from a database
- SQL can create new databases
- SQL can create new tables in a database
- SQL can create stored procedures in a database
- SQL can create views in a database
- SQL can set permissions on tables, procedures, and views

Sci-Πagorus
ıerved.



From Apps to Databases: How Data Flows

At the heart of modern systems is data. Applications communicate with **backend databases**, which are often hosted in the **cloud**.

SQL acts as the bridge between applications and data storage. Through SQL, applications can:

- Retrieve (fetch) data
- Insert (store) data
- Update existing data
- Delete data

This interaction typically follows a flow: **Data Sources → SQL Queries → Destinations**

Data sources may include:

- Relational databases
- Excel files
- Websites
- Cloud systems

Destinations can be:

- Databases
- Dashboards
- OneDrive or Google Drive storage

SQL in Plain Language: What It Is and Why It Matters

SQL (Structured Query Language) is used to **store, retrieve, and manipulate data** inside relational databases. In this program, the focus is on **Microsoft SQL Server** and **TransactSQL (TSQL)**.

Key characteristics of TSQL:

- Used in SQL Server and Azure SQL Databases
- Declarative (you state *what* you want, not *how* to do it)
- Not casesensitive by default

Understanding Relational Databases

A **relational database** stores data in **tables**, also called **entities**.

Each table consists of:

- **Columns** (also known as attributes or fields)
- **Rows** (also known as records, values, or rowsets)

These terms are often used interchangeably, but they all describe how data is structured and accessed.

To manipulate data in these tables, we use **queries**.

Schemas, Tables, and Naming Things Correctly

A **schema** is a logical container or **namespace** that holds database objects such as tables.

Example:

- Schema: Sales
- Tables: Orders, Customers

These tables are referenced as:

- Sales.Orders
- Sales.Customers

Schemas help prevent **name collisions** and improve organization. You can name schemas anything meaningful to the system.

Working with SQL Server

In SQL Server Management Studio:

- You **highlight** a query to execute it
- The * (asterisk) means *select everything* (all columns or values)

Data Type Conversion: Making Values Compatible

Sometimes data types must be converted.

SQL allows **implicit conversion** when data types are compatible. When explicit conversion is required, functions include:

- CAST / TRY_CAST
- CONVERT / TRY_CONVERT
- PARSE / TRY_PARSE

Example: Converting a numeric value into an integer ensures consistent storage and processing.

VARCHAR vs NVARCHAR:

- VARCHAR uses Englishbased characters
- NVARCHAR supports global languages and Unicode

The SQL CREATE TABLE Statement

The CREATE TABLE statement is used to create a new table in a database.

Syntax

```
CREATE TABLE table_name (
    column1 datatype,
    column2 datatype,
    column3 datatype,
    ....
);
```

The column parameters specify the names of the columns of the table.

The datatype parameter specifies the type of data the column can hold (e.g. varchar, integer, date, etc.).

© Cop Sci-Pagorus

Creating Tables: Bringing Structure to Data

To store data, tables must first be created.

General structure:

- Create a database
- Create tables inside schemas

Tables define column names, data types, and constraints, forming the foundation of reliable data storage.

Data Normalization: Keeping Data Clean

Data normalization is the process of organizing data to reduce redundancy and prevent anomalies.

Key forms:

- **First Normal Form (1NF):** No repeating or redundant data in the same row
- **Second Normal Form (2NF):** All nonkey attributes depend on the full primary key
- **Third Normal Form (3NF):** No transitive dependencies

Normalization ensures data remains **accurate, consistent, and trustworthy**.

Relationships between tables exist through **primary and foreign keys**. When a primary key is deleted, related dependent data may also be affected.

Data types

Exact numeric	Approximate numeric	Character	Date/time	Binary	Other
tinyint	float	char	date	binary	cursor
smallint	real	varchar	time	varbinary	hierarchyid
int		text	datetime	image	sql_variant
bigint		nchar	datetime2		table
bit		nvarchar	smalldatetime		timestamp
decimal/numeric		ntext	datetimeoffset		uniqueidentifier
numeric					xml
money					geography
smallmoney					geometry

- Compatible data types can be implicitly converted
- Explicit conversion requires an explicit conversion function:
  CAST / TRY_CAST
  CONVERT / TRY_CONVERT
  PARSE / TRY_PARSE
  STR

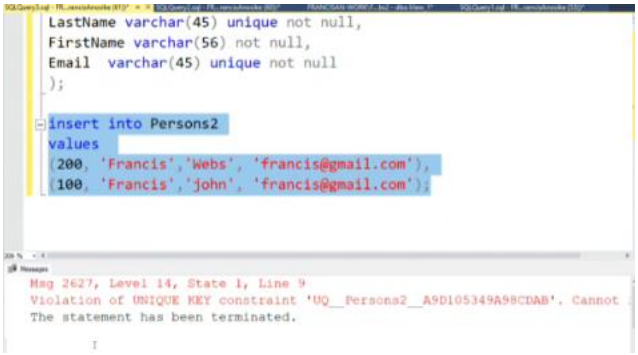
© Copyright Microsoft Corporation. All rights reserved.

Data Types and Constraints: Rules That Protect Your Data

Every column in a table has a **data type**, which defines what kind of data it can store.

Common rules are enforced using **constraints**, such as:

- **NOT NULL** – a value must be provided
- **PRIMARY KEY** – uniquely identifies a row
- **FOREIGN KEY** – creates relationships between tables
- **CHECK** – enforces conditions (e.g. Age >= 18)
- **UNIQUE**- Ensures that all values in a column (or combination of columns) are different.



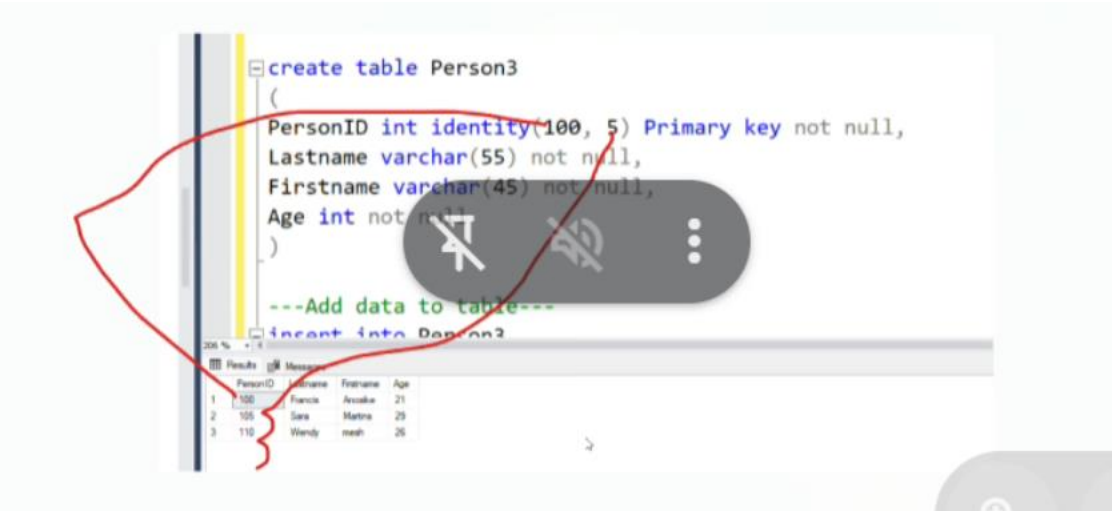
Constraints exist to protect **data integrity**.

AUTOINCREMENTS

Automatically assigns an ID

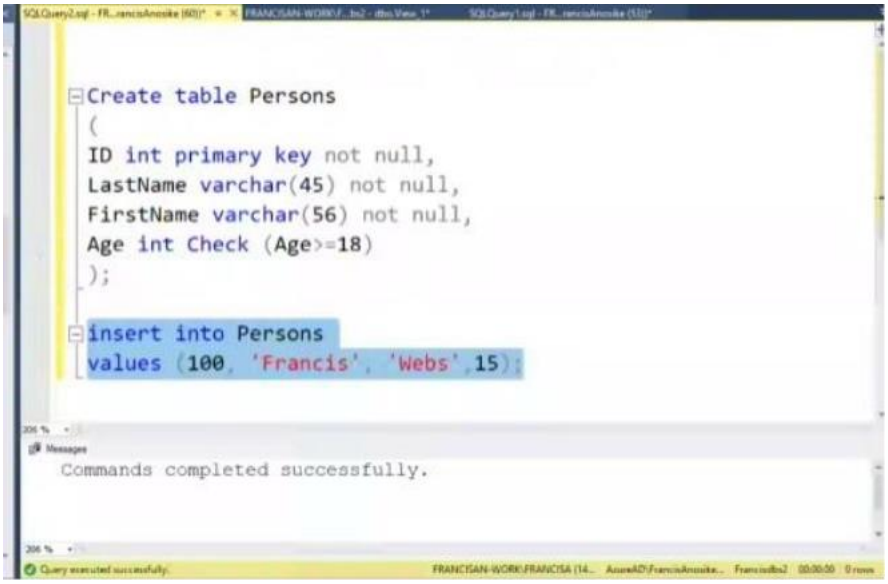
```
create table Person3
(
PersonID int identity(100, 5) Primary key not null,
Lastname varchar(55) not null,
Firstname varchar(45) not null,
Age int not null
)
```

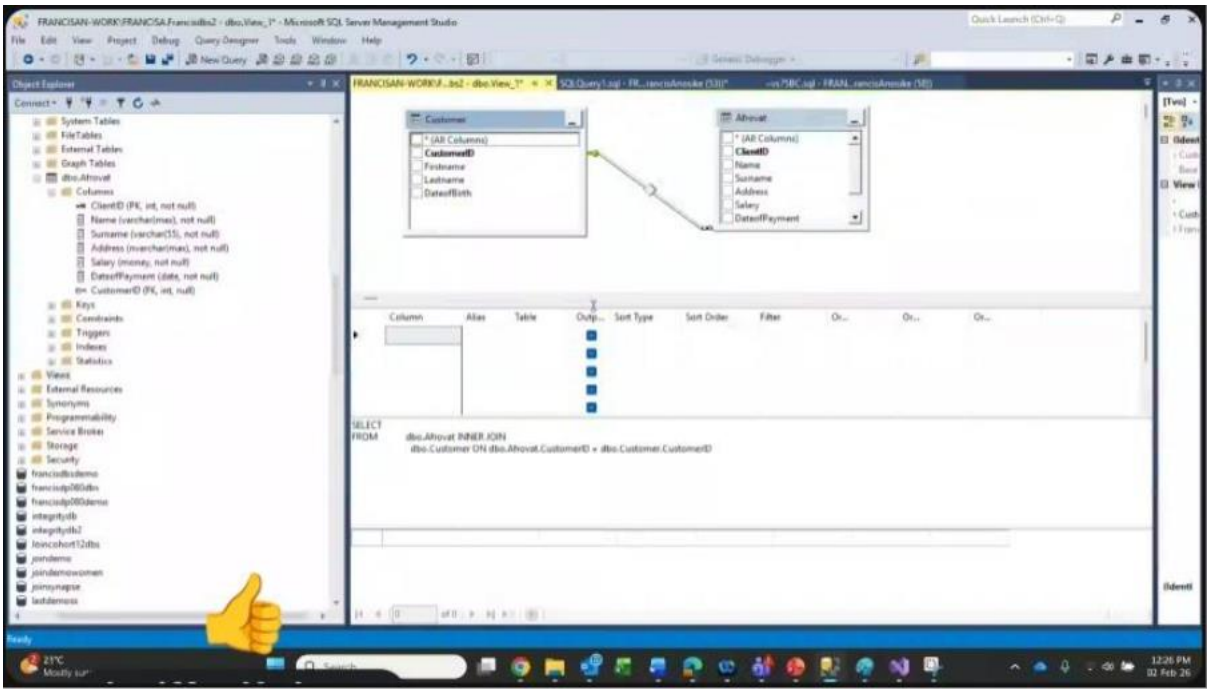
```
---Add data to table---
insert into Person3
values ('Francis', 'Anosike', 21),
```



*ID is 100 and it was incrementing by 5.

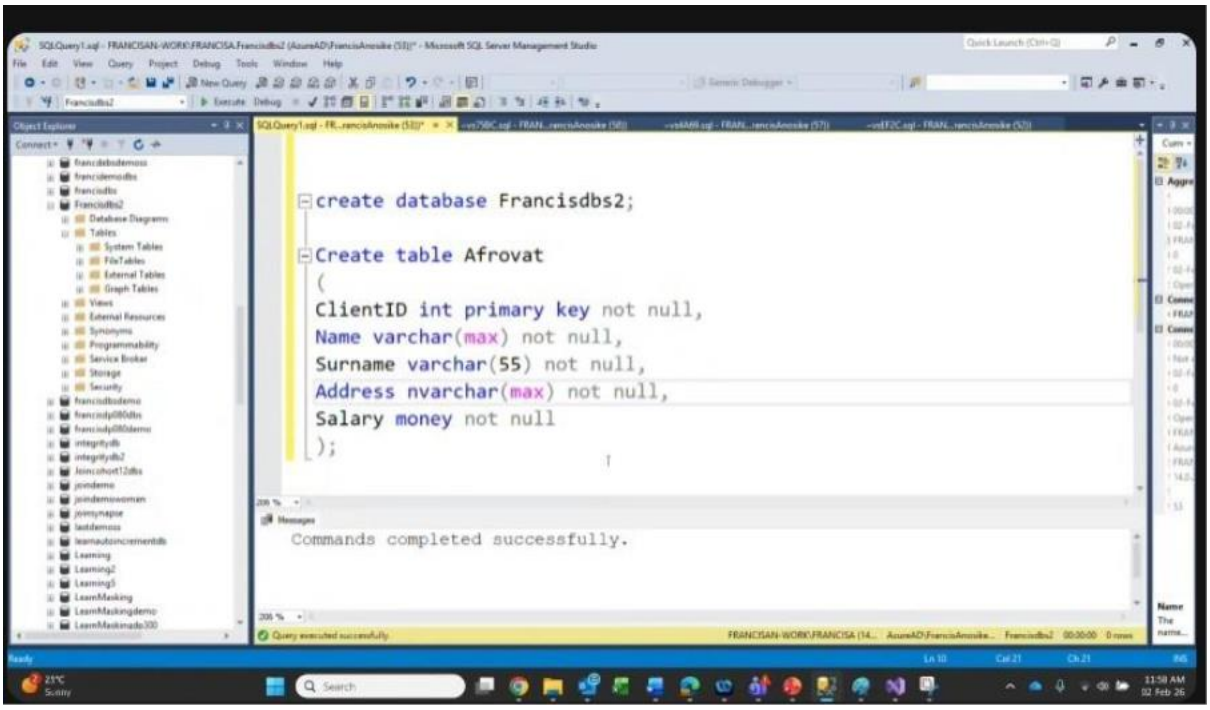
ADDITIONAL INFORMATION





```sql
Create table Customer
(
CustomerID int Primary key not null,
Firstname varchar(45) not null,
Lastname varchar(45) not null,
DateofBirth date not null
)

alter table Afrovat
add foreign key (CustomerID)
references Customer (CustomerID)
```

```sql
);
insert into Afrovat
values (100,'Francis','Anosike', 2000, '02/02/2026');
```



SQL STATEMENT TYPES

SQL statement types

Data Manipulation Language (DML)	Data Definition Language (DDL)	Data Control Language (DCL)
Statements for querying and modifying data: • SELECT • INSERT • UPDATE • DELETE	Statements for defining database objects: • CREATE • ALTER • DROP	Statements for assigning security permissions: • GRANT • REVOKE • DENY

Focus of this course

Sci-Πagorus ed.

DML (Data Manipulation Language) – Query and manipulate

SELECT

INSERT

UPDATE

DELETE (Deletes one row at time)

SQL Queries

- **SELECT**, **INSERT**, **UPDATE**, and **DELETE** statements are the four main types of SQL statements used to manipulate SQL Server data:

- **SELECT** statement: retrieves data
- **INSERT** statement: adds new data
- **UPDATE** statement: modifies data
- **DELETE** statement: deletes data

1.Select:

```
--Retrieve all data person3 table
select * from Person3

select Lastname, Age from Person3 where PersonID =100
```

3.Update:

```
update Person3 set Lastname = 'Thabo', Firstname = 'williams', Age =
```

4.Delete:

```
--Removes data from the Person3 table--
delete from Person3 where PersonID = 100
```

DDL (DATA DEFINITION LANGUAGE)

CREATE

ALTER

DROP (Deletes the entire table)

*Focus on this course is DML & DDL

TRUNCATE TABLE:

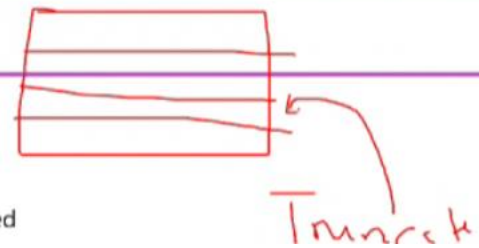
*Removes all the data one at a time at once, but maintains the table.

```
--Truncate removes all the data in the table at once--
Truncate table Person3
```

DELETE removes rows that match the WHERE predicate

- Caution: DELETE without a WHERE clause deletes all rows!

```
DELETE FROM Production.Product
WHERE discontinued = 1;
```



TRUNCATE TABLE clears the entire table

- Storage physically deallocated, rows not individually removed

<u>SQL Operators</u>

Arithmetic Operators

SQL Operators

SQL Arithmetic Operators

Operator	Description
+	Add
-	Subtract
*	Multiply
/	Divide
%	Modulo

<u>e.g.</u>

select 30 + 20

select 30 - 20

select 30 * 20

select 30 / 20

select 2 % 5

<u>Comparison operators</u>

SQL Comparison Operators

Operator	Description
=	Equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to
<>	Not equal to

Equal to:

```
select * from Person3
where Age = 21
```

Greater than or Equal to

```
select * from Person3
where Age >=21
```

Less than or Equal to

Less than

```
select * from Person3
where Age < 29
```

or greater than

Not equal to

```
select * from Person3
where Age <> 29
```

Example:

```
SELECT * FROM Products
WHERE Price = 18;
```

```
SELECT * FROM Products
WHERE Price > 30;
```

```
SELECT * FROM Products
WHERE Price < 30;
```

```
SELECT * FROM Products
WHERE Price >= 30;
```

```
SELECT * FROM Products
WHERE Price <= 30;
```

```
SELECT * FROM Products
WHERE Price <> 18;
```

Logical operators

SQL Logical Operators

Operator	Description
ALL	TRUE if all of the subquery values meet the condition
AND	TRUE if all the conditions separated by AND is TRUE
ANY	TRUE if any of the subquery values meet the condition
BETWEEN	TRUE if the operand is within the range of comparisons
EXISTS	TRUE if the subquery returns one or more records
IN	TRUE if the operand is equal to one of a list of expressions
LIKE	TRUE if the operand matches a pattern
NOT	Displays a record if the condition(s) is NOT TRUE
OR	TRUE if any of the conditions separated by OR is TRUE

Example AND:

```
select * from Person3
where Lastname = 'Francis' AND Firstname = 'Anosike'
```

Both have to match

Example OR:

```
select * from Person3
where Lastname = 'Francis' or Firstname = 'bush'
```

One or the either have to match

Example LIKE:

```
select * from Person3
where Lastname like 'F%'
```

Key pulling

F% (Before)

%F (After)

For search term

The SELECT statement

Element	Expression	Role
SELECT	<select list>	Defines which columns to return
FROM	<table source>	Defines table(s) to query
WHERE	<search condition>	Filters rows using a predicate
GROUP BY	<group by list>	Arranges rows by groups
HAVING	<search condition>	Filters groups using a predicate
ORDER BY	<order by list>	Sorts the output

© Copyright Microsoft Corporation. All rights reserved.

```
SELECT OrderDate, COUNT(OrderID) AS Orders
FROM Sales.SalesOrder
WHERE Status = 'Shipped'
GROUP BY OrderDate
HAVING COUNT(OrderID) > 1
ORDER BY OrderDate DESC;
```

Basic SELECT query examples

All columns

```
SELECT * FROM Production.Product;
```

Specific columns

```
SELECT Name, ListPrice
FROM Production.Product;
```

Expressions and aliases

```
SELECT Name AS Product, ListPrice * 0.9 AS SalePrice
FROM Production.Product;
```

© Copyright Microsoft Corporation. All rights reserved.

SORTING

Sorting results

Use ORDER BY to sort results by one or more columns

- Aliases created in SELECT clause are visible to ORDER BY
- You can order by columns in the source that are not included in the SELECT clause
- You can specify ASC or DESC (ASC is the default)

```
SELECT ProductCategoryID AS Category, ProductName
FROM Production.Product
ORDER BY Category ASC, Price DESC;
```

© Copyright Microsoft Corporation. All rights reserved.

Limiting sorted results

Use TOP to limit the number or percentage of rows returned by a query

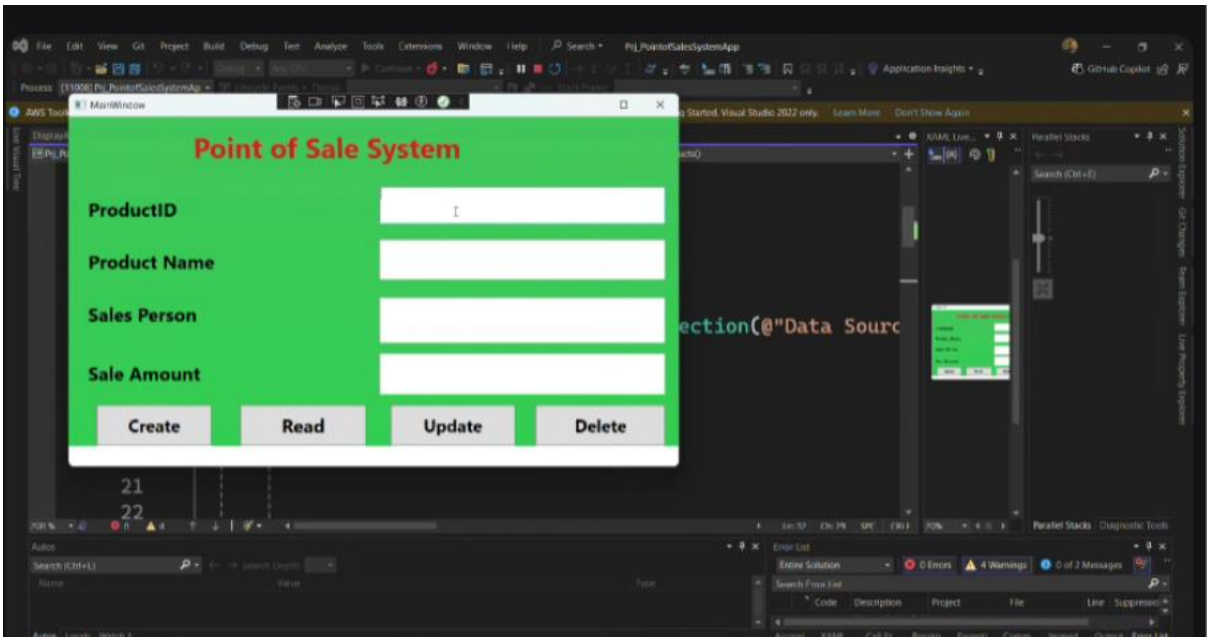
- Works with ORDER BY clause to limit rows by sort order
- Added to SELECT clause:

```
SELECT TOP N [Percent] [WITH TIES]
```

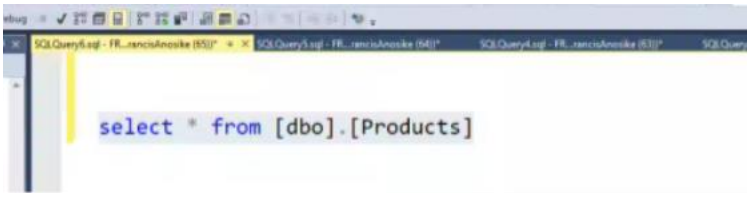
```
SELECT TOP 10 Name, ListPrice
FROM Production.Product
ORDER BY ListPrice DESC;
```

© Copyright Microsoft Corporation. All rights reserved.

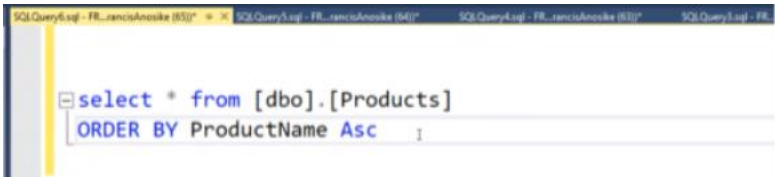
***THEY WORKED ON AN EXAMPLE W ANOTHER CLASS**



Using the select, we could retrieve the product



Order by (categorise this in ascending or discending):



Limiting sorted results

Use TOP to limit the number or percentage of rows returned by a query

- Works with ORDER BY clause to limit rows by sort order
- Added to SELECT clause:

```
SELECT TOP N [Percent] [WITH TIES]
```

```
SELECT TOP 10 Name, ListPrice
FROM Production.Product
ORDER BY ListPrice DESC;
```

© Copyright Microsoft Corporation. All rights reserved.

Paging through results

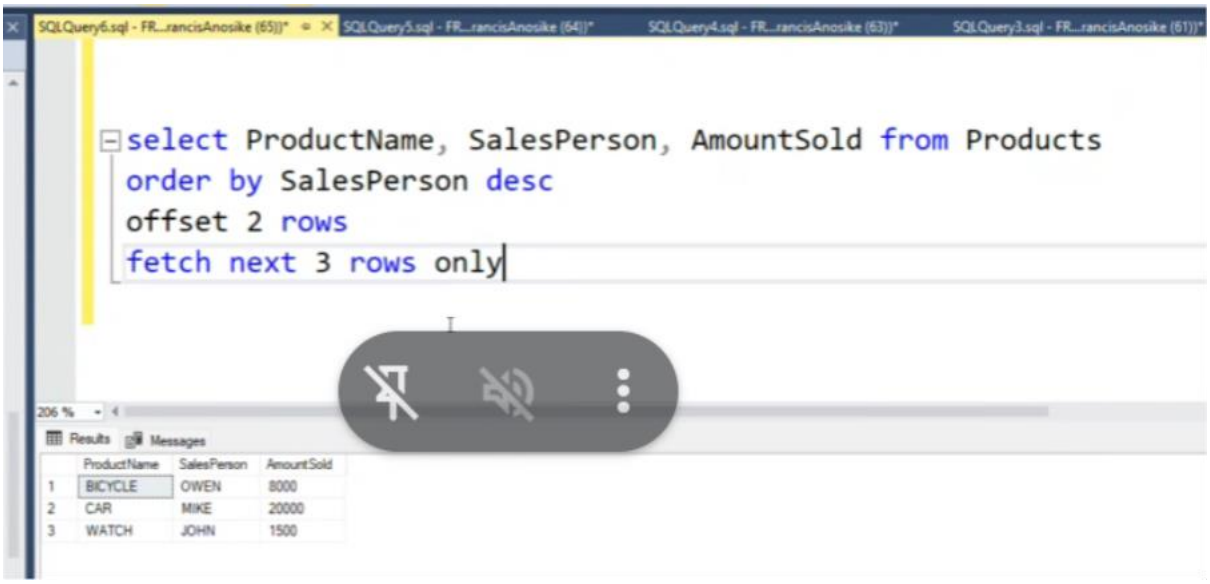
OFFSET-FETCH is an extension to the ORDER BY clause:

- Allows returning a requested range of rows
- Provides a mechanism for paging through results
- Specify number of rows to skip, number of rows to retrieve

```
SELECT ProductID, ProductName, ListPrice
FROM Production.Product
ORDER BY ListPrice DESC
   OFFSET 0 ROWS -- Skip zero rows
   FETCH NEXT 10 ROWS ONLY; -- Get the next 10
```

© Copyright Microsoft Corporation. All rights reserved.

s



Filtering and using predicates

```
SELECT ProductCategoryID AS Category, ProductName
FROM Production.Product
WHERE ProductCategoryID = 2
   AND ListPrice < 10.00
ORDER BY Category, Price DESC;
```

Predicates and Operators	Description
= < >	Compares values for equality/non-equality
IN	Determines whether a specified value matches any value in a subquery or a list
BETWEEN	Specifies an inclusive range to test
LIKE	Determines whether a specific character string matches a specified pattern, which can include wildcards
AND	Combines two Boolean expressions and returns TRUE only when both are TRUE
OR	Combines two Boolean expressions and returns TRUE if either is TRUE
NOT	Reverses the result of a search condition

© Copyright Microsoft Corporation. All rights reserved.

X

TUESDAY (03/FEB/2026) - SQL PRACTICALS

How to run a query

1. Empty Canvas

2. Highlight the query

3. Click Execute after

4. Do not work on the 'master' database, create your own

Here's the example of the Database:

This includes the use of the create, and the insert as well as the select function.

TEMPLATE

xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

```
create database sagedbs


create table Mbulelotable
(
SAID int primary key not null,
Name varchar (45) not null,
Surname varchar (45) not null,
Address varchar (45) not null,
Stipend money not null,
Maritalstatus varchar (45) not null,
Salarypoint int not null
)


--you're asking the select statement to retrieve something--
select * from Mbulelotable


insert into Mbulelotable
```

values (062283, 'Samm', 'Mtshu', 'Braam',10000,'single',12),

(33796, 'Mbulelo', 'Carn', 'Springs',9000,'Married', 13);

--delete data from the table--

delete from Mbulelotable where SAID = 062283

delete from Mbulelotable where SAID = 33796

--update the data in the Mbulelotable--

update Mbulelotable set Name = 'Mbulelos' where SAID = 062283

truncate table Mbulelotable

--needs more work--

select * from Mbulelotable

where Salarypoint < 13

xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

--drop table "nameoftable" deletes the entire table.--

--truncate table "nameoftable" removes everything inside the table but the table is kept--

-- alter table "nameoftable" to add a column--

e.g.

alter table "nameoftable"

add Price int not null

XXXX

- DML (Data manipulation language)
- DDL (Data definition language)
- DCL (Data control language)

<u>DML (Data manipulation language) - works with data inside the table</u>

SELECT- retrieve data from table

INSERT – add data to table

UPDATE – modify data inside the table

DELETE (Deletes one row at time) - delete data in a table

Some are {Truncate}

```
delete from fnb.Gauteng where CustomerID = 100
```

<u>DDL (DATA DEFINITION LANGUAGE) - Works with database objects like tables, views and such</u>

CREATE – create database objects

ALTER – modify database object

DROP (Deletes the entire table) - Remove database object such as a table.

<u>DCL (Data control language) limits access to the database</u>

GRANT – Grant Permission on database objects

REVOKE – take away your Permission on database objects

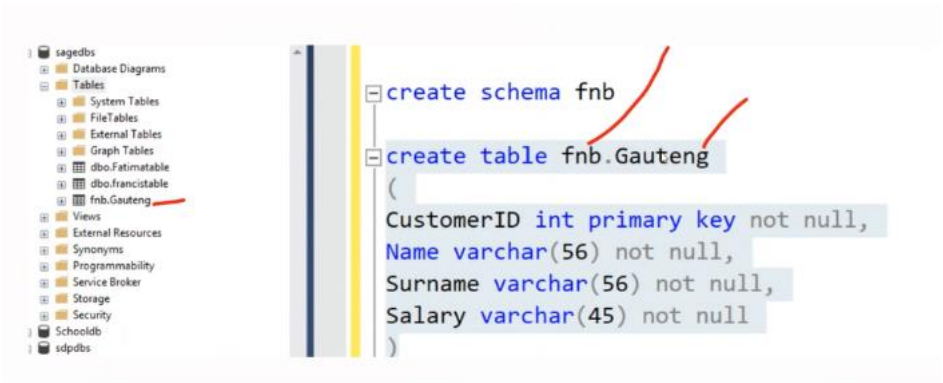
DENY - Deny Permission on database objects

*In this course we won't be using the DCL as much, we will use DML and DDL as DLC is used from database administrators.

<u>SCHEMAS & OBJECTS AGAIN</u>

Schema creates boundaries, is it a constraint though?

e.g.



Avoids having the same database objects, like the same tables and such.

Data types

Exact numeric	Approximate numeric	Character	Date/time	Binary	Other
tinyint	float	char	date	binary	cursor
smallint	real	varchar	time	varbinary	hierarchyid
int		text	datetime	image	sql_variant
bigint		nchar	datetime2		table
bit		nvarchar	smalldatetime		timestamp
decimal/numeric		ntext	datetimeoffset		uniqueidentifier
numeric					xml
money					geography
smallmoney					geometry

- Bigint 64bits (Large range of whole numbers for large IDs and such)
- Int 32bits (medium range of whole numbers)
- Decimal (is for decimal numbers)
- Money (for financial values)
- Varchar (variable character, alphanumeric) = varchar (number of characters)

When updating we use SET:

```
(
CustomerID int primary key not null,
Name varchar(56) not null,
Surname varchar(56) not null,
Salary Money not null
)
insert into fnb.Gauteng
values (100, 'Francis', 'Anosike', 5000);

select * from fnb.Gauteng
update fnb.Gauteng set Name = 'SARA', Surname = 'Martins'
where CustomerID = 100
```

ALTER (DDL) Example:

alter table "nameoftable" add "Price" int not null (to add column)

CLASS EXAMPLE
create schema fnb create table fnb.Gauteng (CustomerID int primary key not null, Name varchar(56) not null, Surname varchar(56) not null, Salary Money not null) insert into fnb.Gauteng values (100, 'Francis', 'Anosike', 5000), (200, 'WENDY', 'MESH', 3000), (300, 'JOHN', 'WEBS', 6000); select * from fnb.Gauteng update fnb.Gauteng set Name = 'SARA', Surname = 'Martins', Salary = 1000 where CustomerID = 100 delete from fnb.Gauteng where CustomerID = 100 truncate table fnb.Gauteng DROP table fnb.Gauteng Alter table fnb.Gauteng add Price int not null

MORE EXAMPLES (Connected to the above)

LOGICAL OPERATORS (LIKE) - can work with numbers as well.

Select * from fnb.Gauteng where Name like ‘F%’ - names that start with F, Francis.
Select * from fnb.Gauteng where Name like ‘%Y’ - names that end with Y, Wendy.
Select * from fnb.Gauteng where CustomerID like ‘2%’ - ID’s that start with 2, Wendy.
Select * from fnb.Gauteng where CustomerID like ‘%0’ - ID’s that end with 0, Wendy.

LOGICAL OPERATORS (BETWEEN) - This is to help get ranges of things
Select * from fnb.Gauteng
Where Salary between 5000 and 6000 – We can use this to determine high salary earners
LOGICAL OPERATORS (AND/OR) -
Select from fnb.Gauteng
where CustomerID = 500 OR Name = 'WENDY’ - OR operator chooses any true condition to display.
Select from fnb.Gauteng
where CustomerID = 200 AND Name = WENDY – AND operator declares all conditions must be true to display

COMPARISON OPERATORS – they’re like conditional operators in coding.
Select * from fnb.Gauteng
Where Salary >= 5000 – We can use this to determine salary earners who earn higher than R5000 (Greater or equal to)
Select * from fnb.Gauteng
Where Salary <= 6000 – We can use this to determine salary earners who earn lower than R6000 (Less than or equal to)

ARITHMETIC OPERATORS
Select 30 * 4
Select SQRT (16)
Select LOG10 (10)

ADDITIONAL OPERATORS

Select UPPER (StaffName) from Sales – To convert to uppercase
Select LOW (StaffName) from Sales – To convert to uppercase
Select LEN ('Mbulelo’) – To count length of characters
Select Replace ('Mbulelo’, ‘u’,’a’) – To replace characters from a word
Select Concat ('Francis',' ', 'Anosike') as Fullnames – For Space
TIME RELATED:
Select month ('2026/02/04') as Months
Select Year ('2026/02/04') as Years
Select Day ('2026/02/04') as Dayss
Select SYSDATETIME() as Result
Select GETUTCDATE() as Result
Select IIF (500 < 1000, ‘Yes’, ‘No’) as Result – Displaying condition

Check constraint (Helps for condition setting):

(Age verification example)

```
create table Persons
(
ID int primary key not null,
Lastname varchar(56) not null,
Firstname varchar(56) not null,
Age int Check (Age >= 18)
)
```

My Example of the check constraint:

Check constraint 1:

```
create database sagedbs
create table persons
(
ID int primary key not null,
Firstname varchar (60) not null,
Lastname varchar (60) not null,
Age int check (Age>=18)
)

insert into persons
values (100, 'Mbulelo','Mtshude', 19),
(200, 'Sam','Mabiza', 18);

select * from persons
```

Check constraint 2:

SQLQuery1....-User (53))*

```
create database sagedbs
create table persons
(
ID int primary key not null,
Firstname varchar (60) not null,
Lastname varchar (60) not null,
Code varchar (60) check (Code = '1XQ')
)

insert into persons
values (100, 'Mbulelo','Mtshude', '1xq'),
(200, 'Sam','Mabiza', '1xq');

select * from persons
```

```
create database sagedbs
create table persons
(
ID int primary key not null,
```

```
Firstname varchar (60) not null,
Lastname varchar (60) not null,
Code varchar (60) check (Code = '1XQ')
)

insert into persons
values (100, 'Mbulelo','Mtshude', '1xq'),
(200, 'Sam','Mabiza', '1xq');

select * from persons
```

Unique constraint (all values in the column values are different)

```
create table persons

(

ID int primary key not null,

Firstname varchar (60) unique not null,

Lastname varchar (60) unique not null,

Code varchar (60) check (Code = '1XQ') not null

)



insert into persons

values (100, 'Mbulelo','Mtshude', '1xq'),
```

(200, 'Samuel','Mabiza', '1xq');

select * from persons

drop table persons

Autoincrements constraint (setting a condition to increase ID generation automatically)

It works like (in blue): identity (starting number, increment number), then it automatically generates more ID codes by the number of increments, i.e. if you made your increment to be 7, and the ID 10, it will add the 7 to the 10, on the next ID it will be 17 (so 10 + 7 = 17), then 17 + 7 = 24, so your ID's are 10, 17 and 24.

create table person

(

ID int identity (100,05) primary key not null,

Firstname varchar (60) unique not null,

Lastname varchar (60) unique not null,

Age int not null

)

insert into person

values ('Mbulelo','Mtshude', 18),

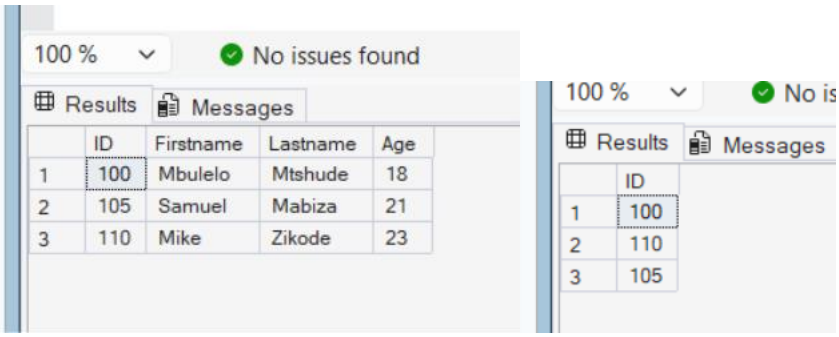
('Samuel','Mabiza', 21),

('Mike','Zikode', 23);

select * from person

drop table person

RESULT (ID Column):



Aggregate functions (for calculations):

Sum, AVG, Max, Min & Count (We use it with the select function)

Create table Sales(

staffID int identity (110,5) primary key not null,

staffName varchar (45) not null,

staffLastName varchar (45) not null,

Department varchar (45) not null,

Itemsold varchar (45) not null,

Amount money not null,

Quantity int not null,

)

insert into Sales

values ('Mbulelo','Mtshude','sales department', 'TV', 16000, 1),

('Mike','Zikode','Sales Department', 'Iphone 17', 25000, 3),

('Wendy', 'Sphara', 'Admin Department', 'Car', 150000, 2),

('John', 'Webbs', 'Admin Department', 'Shoes', 1000, 5),

('Samuel', 'Mabiza', 'Sports Department', 'Soccer Shoes', 25000, 5);

```
--total of amount sold--
select Sum(Amount) from Sales

--average sales/mean--
select Avg(Amount) from Sales

--maximum sales made--
select Max(Amount) from Sales

--minimum sales made--
select Min(Amount) from Sales

--Quantity sales made--
select Count(Quantity) from Sales
```

select * from Sales

drop table Sales

100 % No issues found

Results Messages

	staffID	staffName	staffLastName	Department	Itemsold	Amount	Quantity
1	110	Mbulelo	Mtshude	sales department	TV	16000,00	1
2	115	Mike	Zikode	Sales Department	Iphone 17	25000,00	3
3	120	Wendy	Sphara	Admin Department	Car	150000,00	2
4	125	John	Webbs	Admin Department	Shoes	1000,00	5
5	130	Samuel	Mabiza	Sports Department	Soccer Shoes	25000,00	5

```
SELECT * FROM Products
WHERE Price = 18;
```

```
SELECT * FROM Products
WHERE Price > 30;
```

```
SELECT * FROM Products
WHERE Price < 30;
```

```
SELECT * FROM Products
WHERE Price >= 30;
```

```
SELECT * FROM Products
WHERE Price <= 30;
```

© Copyright Microsoft Corporation. All rights reserved.

meet.google.com is sharing your screen. Stop sharing Hide

```
SELECT * FROM Products
WHERE Price <> 18;
```

Clauses tomorrow

The SELECT statement

Element	Expression	Role
SELECT	<select list>	Defines which columns to return
FROM	<table source>	Defines table(s) to query
WHERE	<search condition>	Filters rows using a predicate
GROUP BY	<group by list>	Arranges rows by groups
HAVING	<search condition>	Filters groups using a predicate
ORDER BY	<order by list>	Sorts the output

© Copyright Microsoft Corporation. All rights reserved.

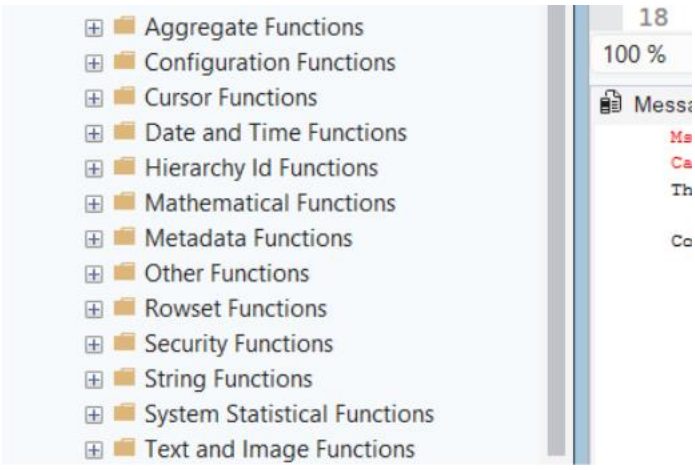
```
SELECT OrderDate, COUNT(OrderID) AS Orders
FROM Sales.SalesOrder
WHERE Status = 'Shipped'
GROUP BY OrderDate
HAVING COUNT(OrderID) > 1
ORDER BY OrderDate DESC;
```

meet.google.com is sharing your screen. Stop sharing Hide

meet.google.com
FRANCIS AN...
The SELECT statement
Mbulelo Mtshuts

SIMPLE CHEAT TO ACCESS FUNCTIONS

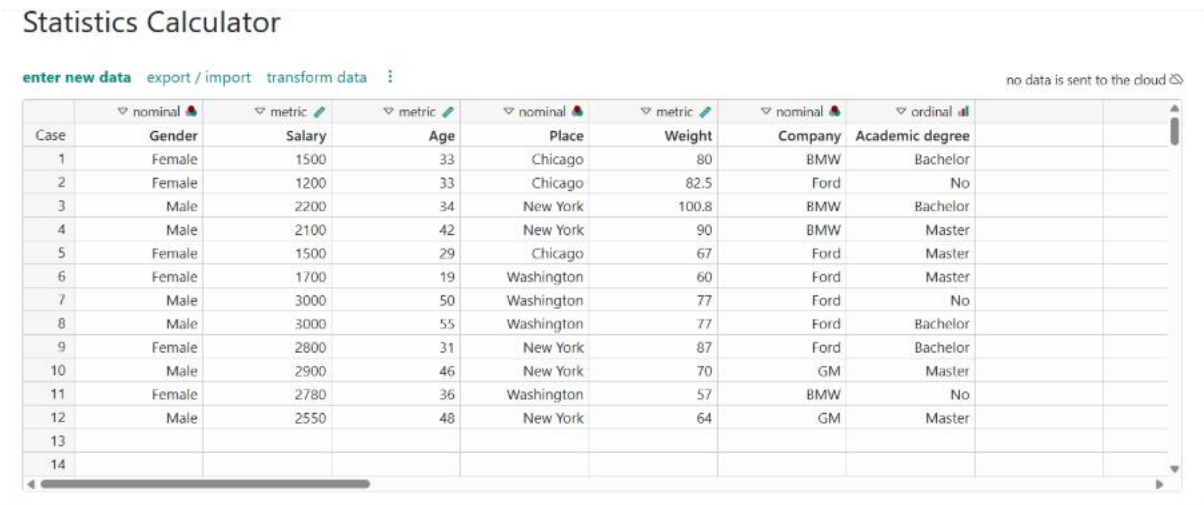
- **your database > programmability>System Functions**



HOMEWORK

https://numiqo.com/statistics-calculator/descriptive-statistics

Create the table exactly as it shows using your own code for practice



	nominal	metric	metric	nominal	metric	nominal	ordinal
Case	Gender	Salary	Age	Place	Weight	Company	Academic degree
1	Female	1500	33	Chicago	80	BMW	Bachelor
2	Female	1200	33	Chicago	82.5	Ford	No
3	Male	2200	34	New York	100.8	BMW	Bachelor
4	Male	2100	42	New York	90	BMW	Master
5	Female	1500	29	Chicago	67	Ford	Master
6	Female	1700	19	Washington	60	Ford	Master
7	Male	3000	50	Washington	77	Ford	No
8	Male	3000	55	Washington	77	Ford	Bachelor
9	Female	2800	31	New York	87	Ford	Bachelor
10	Male	2900	46	New York	70	GM	Master
11	Female	2780	36	Washington	57	BMW	No
12	Male	2550	48	New York	64	GM	Master
13							
14							

Then calculate the following:

	Salary
Mean	2269.17
Std. Deviation	658.61
Minimum	1200
Maximum	3000
Number of values	12

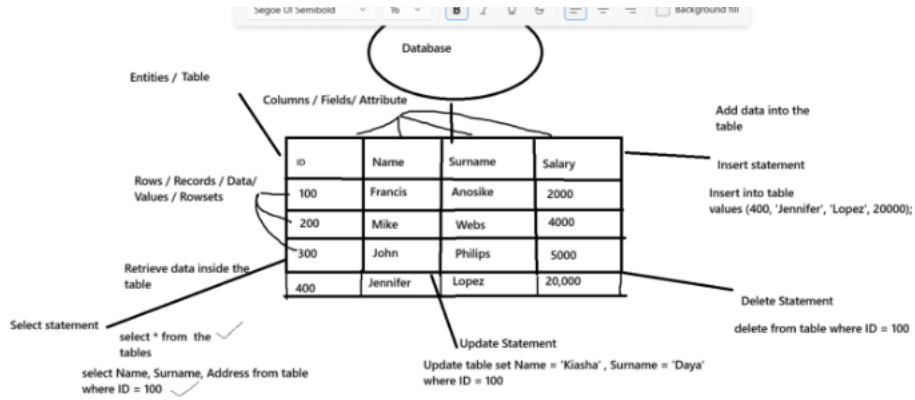
Here's the homework

```
create table Homework
(
CaseID int identity (1,01) primary key not null,
PersonName varchar(50) not null,
Gender varchar(10) not null,
Salary money not null,
Age int not null,
Place varchar (50),
Weight decimal not null,
Company varchar (50)not null,
AcademicDegree varchar (50) not null
)

insert into Homework
values
('Bryce','Female',1500,33,'Chicago',80, 'BMW', 'Bachelor'),
('Lorato','Female',1200,33,'Chicago',82.5, 'Ford', 'No'),
('Bruce','Male',2200,34,'New York',100.8, 'BMW', 'Bachelor'),
('Steve','Male',2100,42,'New York',90, 'BMW', 'Master'),
('Sabrina','Female',1500,29,'Chicago',67, 'Ford', 'Master'),
('Indigo','Female',1700,19,'Washington',60, 'Ford', 'Master'),
('Sfiso','Male',3000,50,'Washington',77, 'Ford', 'No'),
('Sam','Male',3000,55,'Washington',77, 'Ford', 'Bachelor'),
('Lucy','Female',2800,31,'New York',87, 'Ford', 'Bachelor'),
('Ernest','Male',2900,46,'New York',70, 'GM', 'Master'),
('Jackie','Female',2780,36,'Washington',57, 'BMW', 'No'),
('Jason','Male',2550,48,'New York',64, 'GM', 'Master');

select * from Homework
```

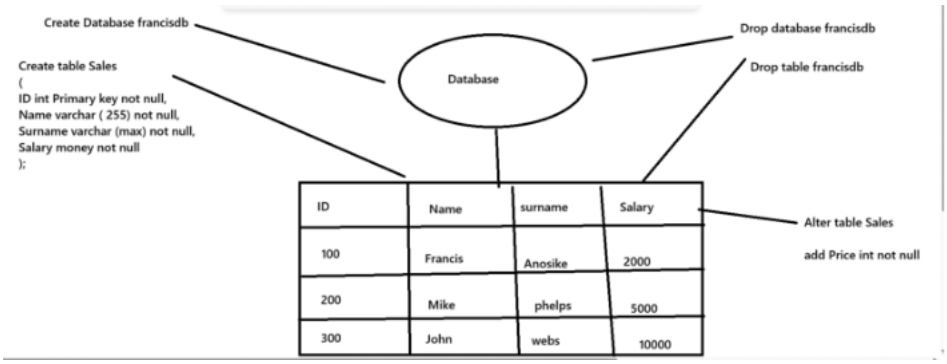
```
Select Avg(Salary) from Homework
Select STDEV (Salary) from Homework
Select Min(Salary) FROM Homework
Select Max (Salary) FROM Homework
Select Count (PersonName) FROM Homework

select * from Homework
```

How to import the data from the database to Excel from SSMS

THURSDAY (05/FEB/2026) - Session Recap (of everything we did)



DDL explanations again:



<u>Class Example (Repeat)</u>:

Create table Sales

(

ID int Primary key not null,

Name varchar (255) not null,

Surname varchar (max) not null,

Salary money not null

)

insert into Sales

values (100, 'Francis','Anosike',2000),

(200,'Mike','phelps',5000),

(300,'John','webs',10000);

select * from Sales

Alter table Sales

add Price int not null

Drop database Sales

Drop table Sales

Logical order of Query processing

Logical Order OF Query Processing

5 SELECT
1 FROM
2 WHERE Operates on rows
3 GROUP BY Creates groups
4 HAVING Operates on groups
6 ORDER BY

1. From
2. Where
3. Group by (Creates groups)
4. Having (works in the groups)
5. Select
6. Order by (Ascending/Descending orders)

*There is an order of precedence or priorities

*Alias is the 'AS'

Expressions and aliases

CS

```
SELECT Name AS Product, ListPrice * 0.9 AS SalePrice
FROM Production.Product;
```

Orderby – Data manipulation clause

Sorting results

— Clause

Use ORDER BY to sort results by one or more columns

- Aliases created in SELECT clause are visible to ORDER BY
- You can order by columns in the source that are not included in the SELECT clause
- You can specify ASC or DESC (ASC is the default)

```
SELECT ProductCategoryID AS Category, ProductName
FROM Production.Product
ORDER BY Category ASC, Price DESC;
```

<u>Here's an example of the orderby clause:</u>

```
create table Product
(
ID int primary key not null,
Name varchar(45) not null,
Item varchar(56) not null,
Amount money not null,
Quantity int not null
)


insert into Product
values
(100, 'TV', 'Tv stand', 300, 1),
```

```
(200, 'shoes', 'collections', 1000, 3),
(300, 'cars', 'automobile', 90000, 6),
(400, 'laptops', 'computer', 13000, 5);
```

--orderby is used to sort out data either in ascending or descending order--

```
select * from Product
order by Name desc
```

```
Alter table Product
add Price int not null
```

```
Drop table Product
```

The GroupBy:

Here's the code:

```
CREATE TABLE Customer (
 CUSTID int primary key not null,
Years varchar(50) not null,
Amount money not null
 )
 insert into Customer values (108, '2010', 100),
(101, '2010', 100), (102, '2010', 200),
 (103, '2011', 300), (104, '2012', 400),
 (105, '2012', 500), (106, '2011', 600),
(107, '2012', 700);
```

select years, sum(Amount) FROM Customer

group by Years

- Groups similar

Having (selects a specific condition):

select sum(Amount), Years FROM Customer

group by Years

Having sum (Amount) > 900

- Groups similar items

Example:

```
CREATE TABLE Customer
(
CUSTID int primary key not null,
Years varchar(50) not null,
Amount money not null
 )
 insert into Customer values (108, '2010', 100),
 (101, '2010', 100),
(102, '2010', 200),
(103, '2011', 300),
(104, '2012', 400),
(105, '2012', 500),
(106, '2011', 600),
(107, '2012', 700);
```

--group by clause--

select sum(Amount) Years FROM Customer group by Years

--Having clause--

select sum(Amount) Years FROM Customer group by Years

Having sum (Amount) <= 900

Limit sorted results:

*In a list of items, you can limit how many items to show, eg. From 5 items in the list display only 3 items. By using the keyword “top…”.

select top 2 * from Customer

xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

-Skip what you want to skip and fetch what you want.

E.g:

Or

CREATE TABLE Customer

(

CUSTID int primary key not null,

Years varchar(50) not null,

Amount money not null

)

insert into Customer

```
values (108, '2010', 100),
(101, '2010', 100),
(102, '2010', 200),
(103, '2011', 300),
(104, '2012', 400),
(105, '2012', 500),
(106, '2011', 600),
(107, '2012', 700);

SELECT * FROM Customer
ORDER BY CUSTID
OFFSET 2 ROWS      -- Skips the first 2 rows--
FETCH NEXT 3 ROWS ONLY; -- Retrieves the next 3 rows--
```

MODULE 2

How to identify a FUNCTION:
-The purple color followed by a parenthesis.

e.g. **Select Avg () from Table**

SIMPLE CHEAT TO ACCESS FUNCTIONS

- **your database > programmability>System Functions**

SQL Sum function syntax (If you want, you can add the where function):

CONCATINATION FUNCTION:

Only for strings and not numbers.

*Numbers SUM Function, String Concatenation Function:

THE REST ARE HERE PG 25

SQL Server REPLACE() Function

The REPLACE() function replaces all occurrences of a substring within a string, with a new substring.

Example

Replace "SQL" with "HTML":

```
SELECT REPLACE('SQL Tutorial', 'SQL', 'HTML');
```

```
select Replace ('Philip', 'P', 'S')
```

The IIF FUNCTION (IF CONDITION):

```
select IIF (Salary >= 6000, 'Good Salary Earners', 'Poor Salary' )
from fnb.Gauteng
```

206 %

Results | Messages

	(No column name)
1	Poor Salary
2	Poor Salary
3	Good Salary Earners

The System Time:

SQL Server SYSDATETIME() Function

Syntax

```
SYSDATETIME()
```

Example

Return the date and time of the SQL Server:

```
SELECT SYSDATETIME() AS SysDateTime;
```

(You can find the in the files)

Select GETDATE()

--I want to track my Company Projects. Each project has a unique ID, a name, a budget, and a start date. --I also need to know which Publishing Partner is assigned to each project. --I need to see a list of all projects started in 2026 with a budget over R100,000, --but I only want to see the Project Name and the Partner's name.

FRIDAY (06/FEB/2026) - CASE FUNCTION/Decisions

For decision making in SQL you can use either the IIF Function, but also the CASE function:

The SQL CASE Expression

The CASE expression goes through conditions and returns a value when the first condition is met (like an if-then-else statement). So, once a condition is true, it will stop reading and return the result. If no conditions are true, it returns the value in the ELSE clause.

If there is no ELSE part and no conditions are true, it returns NULL.

```
CASE
    WHEN condition1 THEN result1
    WHEN condition2 THEN result2
    WHEN conditionN THEN resultN
    ELSE result
END;
```

This is used with multiple conditions at a time.

EXAMPLE:

Example

```
SELECT OrderID, Quantity,
CASE
    WHEN Quantity > 30 THEN 'The quantity is greater than 30'
    WHEN Quantity = 30 THEN 'The quantity is 30'
    ELSE 'The quantity is under 30'
END AS QuantityText
FROM OrderDetails;
```

Class example (Case Expression) :

create table SalaryMonitor(

ID int primary key not null,

Name varchar(50) not null,

Salary int not null

)

insert into SalaryMonitor

values

(101,'Syd', 10000),

(201, 'Mary', 25000),

(301, 'Jerry', 7500),

(110,'Alex', 18000),

```
(210, 'Levi', 5400),
(310, 'Nhlanhla', 5000);


select * FROM SalaryMonitor


select ID, Name,
CASE
When Salary < 5500 then 'Below Minimum wage'
When Salary >= 5500 then 'Above Minimum wage'
else 'Poor Salary'
end as Result
FROM SalaryMonitor


drop table SalaryMonitor
```

LIKE OPERATOR- For a pattern recognition:

Used with the WHERE clause, % (one or single character)

e.g:

The pattern recognition to pull up the letter A

```sql
insert into StaffSalary
values
(100, 'Francis', 'Anosike', 5000),
(200, 'Sara', 'Martins', 15000),
(300, 'Wendy', 'Mesh', 50000),
(400, 'Mark', 'Owen', 20000),
(500, 'John', 'Webs', 10000);

select * from StaffSalary
where Name like '%a%'
```

206 %

Results | Messages

	ID	Name	Surname	Salary
1	100	Francis	Anosike	5000.00
2	200	Sara	Martins	15000.00
3	400	Mark	Owen	20000.00

meet.google.com is sharing your screen.

```sql
values
(100, 'Francis', 'Anosike', 5000),
(200, 'Sara', 'Martins', 15000),
(300, 'Wendy', 'Mesh', 50000),
(400, 'Mark', 'Owen', 20000),
(500, 'John', 'Webs', 10000);

select * from StaffSalary
where Name like 'F%' or Name like 'S%'
```

206 %

Results | Messages

	ID	Name	Surname	Salary
1	100	Francis	Anosike	5000.00
2	200	Sara	Martins	15000.00

meet.google.com is sharing your screen. Stop sharing

(You can use AND/OR with the LIKE function), ‘%S’, ‘S%’ or ‘%S%’ to search specific characters.

But also we can:

Example

Return all customers that starts with "b" and ends with "s":

```
SELECT * FROM Customers
WHERE CustomerName LIKE 'b%s';
```

'b%s' starts with B and ends with S.

Contains

To return records that contains a specific letter or phrase, add the % both before and after the letter or phrase.

Example

Return all customers that contains the phrase 'or'

```
SELECT * FROM Customers
WHERE CustomerName LIKE '%or%';
```

xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

***the _ (underscore): WILD CARD**

The _ Wildcard

The _ wildcard represents a single character.

It can be any character or number, but each _ represents one, and only one, character.

Example

Return all customers from a city that starts with 'L' followed by one wildcard character, then 'nd' and then two wildc

```
SELECT * FROM Customers
WHERE city LIKE 'L_nd__';
```

meet.google.com is sharing your screen. Stop sharing Hide

Combine Wildcards

Any wildcard, like % and _ , can be used in combination with other wildcards.

Example

Return all customers that starts with "a" and are at least 3 characters in length:

```
SELECT * FROM Customers
WHERE CustomerName LIKE 'a__%';
```

Second positions:

Example

Return all customers that have "r" in the second position:

```
SELECT * FROM Customers
WHERE CustomerName LIKE '_r%';
```

Without the wildcard, the normal search query would be the exact keyword:

Without Wildcard

If no wildcard is specified, the phrase has to have an exact match to return a result.

Example

Return all customers from Spain:

```
SELECT * FROM Customers
WHERE Country LIKE 'Spain';
```

SQL ALIASES (we give a temporary name to a column, this is not part of the table exactly):

SQL Aliases

SQL aliases are used to give a table, or a column in a table, a temporary name.

Aliases are often used to make column names more readable.

An alias only exists for the duration of that query.

An alias is created with the `AS` keyword.

Example

```
SELECT CustomerID AS ID
FROM Customers;
```

meet.google.com is sharing your screen. Stop sharing Hide

AS is completely optional! Most Devs use this instead:

```
select ID  D, Name  N, Surname S, Salary SA
from StaffSalary
```

Where ID is “D” instead of ID is as “D”.

Dates:

SQL Working With Dates

SQL Server comes with the following data types for storing a date or a date/time value in the database:

- `DATE` - format YYYY-MM-DD
- `DATETIME` - format: YYYY-MM-DD HH:MI:SS
- `SMALLDATETIME` - format: YYYY-MM-DD HH:MI:SS
- `TIMESTAMP` - format: a unique number

COMMENTS:

-- then comment --

Those are single line comments.

These are multi-line comment:

Multi-line Comments

Multi-line comments start with `/*` and end with `*/`.

Any text between /* and */ will be ignored.

The following example uses a multi-line comment as an explanation:

Example

```
/*Select all the columns
of all the records
in the Customers table:*/
SELECT * FROM Customers;
```

meet.google.com is sharing your screen. Stop sharing Hide

RELATIONSHIPS (TYPES)

1️⃣ One-to-One (1 : 1) Relationship

Meaning:

One row in Table A matches **only one** row in Table B.

Example:

A **Person** has only **one Passport**.

```pgsql
+-----------+          +------------+
| Person    |          | Passport   |
+-----------+          +------------+
| PersonID  |<------->| PersonID   |
| Name      |          | PassportNo |
+-----------+          +------------+
     1                      1
```

SQL Idea:

- `Passport.PersonID` is both **PK and FK** (unique relationship).

A table may have a relationship with another table.

2 One-to-Many (1 : M) Relationship

Meaning:

One row in Table A can match **many rows** in Table B.

Example:

A **Customer** can place many **Orders**.

```pgsql
+-------------+          +-------------+
| Customer    |          | Orders      |
+-------------+          +-------------+
| CustomerID  |<------   | CustomerID  |
| Name        |          | OrderID     |
+-------------+          | OrderDate   |
      1                  +-------------+
                               M
```

meet.google.com is sharing your screen. Stop sharing Hide

Key Point:

- `Orders.CustomerID` is a **Foreign Key** referencing `Customer.CustomerID`.

We're adding foreign keys now:

```sql
Alter table Department
add Foreign key (ID)
references StaffSalary (ID)
```

```sql
insert into Department
values
(400, 'HR', 100),
(, 'HR', 100),
(400, 'HR', 100),
(400, 'HR', 100),
(400, 'HR', 100),
(400, 'HR', 100),
```

Here's the class example:

Linking Primary and Foreign Keys:
Create table StaffSalary (ID int primary key not null, Name varchar(45) not null, Surname varchar(56) not null, Salary money not null); insert into StaffSalary values (100, 'Francis', 'Anosike', 5000), (200, 'Sara', 'Martins', 15000), (300, 'Wendy', 'Mesh', 50000), (400, 'Mark', 'Owen', 20000), (500, 'John',

'Webs', 10000); select * from StaffSalary Create table Department (DepartmentID int primary key not null, DpartmentName varchar(45) not null) Alter table Department --add ID int-- --Alter table Department --add Foreign key (ID) --references StaffSalary (ID)-- insert into Department values (400, 'HR', 100), (300, 'Firedrill', 200), (600, 'Receptionist', 300), (900, 'Finance', 400), (800, 'Payroll', 500); select * from Department

- USED POWER BI FOR REPORT CREATION AS AN EXAMPLE

*After Lunch (You Haven an Exercise in your emails).

SQL EXERCISE Answers:

--employee information 1 create table EmployeeTb (EmpID int primary key not null, Name varchar(50) not null, Surname varchar(50) not null, IDNumber char(13) not null, ContactNumber varchar(20) not null, IsManager bit, UserName varchar(20) not null, Password varchar(20) not null,)

insert into EmployeeTb values(01,'John','Jared',0103061111083,0687321919,0,'JohnJ','jared2001'), (02,'Saints','Helena',9912262222081,073542796,1,'SaintsH','helenaisboss'), (03,'Boyd','Jennings',0007203333083,081746669,0,'BoydJen','Jenerally10'), (04,'Becca','Maini',9704304444081,0760025831,0,'BeccaMaini','becca3maini'), (05,'Herald','Govender',0010245555083,0693387200,1,'GovHeralds','herraldings2');

select EmpID,Name, Surname, IDNumber, CASE When IsManager = 0 then 'Employee' When IsManager = 1 then 'Manager*' else 'Not Employed' end as Result FROM EmployeeTb

MONDAY (09/FEB/2026) - JOINS

Ronald

Adding Columns To The table in the sales schema

CREATE SCHEMA sales;
 	
CREATE TABLE sales.quotations
(
  quotation_no INT IDENTITY PRIMARY KEY,
  valid_from DATE NOT NULL,
  valid_to DATE NOT NULL
);
 	
select * From sales.quotations
 	
ALTER TABLE sales.quotations
ADD amount DECIMAL (10,2) NOT NULL,
  customer_name VARCHAR (50) NOT NULL;

SQL JOINS

Syntax for the types of joins:

Class Example of Joins:

--Create table actors CREATE TABLE actors (

actor_id INT PRIMARY KEY,

actor_name VARCHAR(100),

movie_id INT

);

---Insert data into actors INSERT INTO actors (actor_id, actor_name, movie_id) VALUES

(1, 'Vin Diesel', 301),

(2, 'Marilyn Monroe', 302),

(3, 'Alvaro Morte', 303),

(4, 'Rose DeWitt', 304),

(5, 'Halle Berry', 306);

```
CREATE TABLE movies (
    movie_id INT PRIMARY KEY,
    movie_title VARCHAR(100)
);

INSERT INTO movies (movie_id, movie_title) VALUES
(301, 'Fast and Furious'),
(302, 'Blonde'),
(303, 'Money Heiast'),
(304, 'Titanic'),
(305, 'John Wick');

SELECT * FROM movies

--query that performs a join operation
SELECT actors.actor_name, movies.movie_title
FROM actors
JOIN movies
ON actors.movie_id = movies.movie_id;
```

ALL THE TYPES OF JOINS:

ANother Example, of Inner Join:

-- Table 1: Employee CREATE TABLE Employee (emp_id INT PRIMARY KEY, name VARCHAR(50) NOT NULL, age INT CHECK (age > 0));

INSERT INTO Employee VALUES (1, 'Akhona', 30), (2, 'Leasha', 25), (3, 'Jennifer', 28), (4, 'Refiloe', 35), (5, 'Mbulelo', 40), (6, 'Josepht', 29), (7, 'Thabiso', 33);

-- Table 2: Departments CREATE TABLE Departments (dept_id INT PRIMARY KEY, emp_id INT, department_name VARCHAR(50) NOT NULL, FOREIGN KEY (emp_id) REFERENCES Employee(emp_id));

INSERT INTO Departments VALUES (101, 1, 'HR'), (102, 3, 'Sales'), (103, 4, 'Marketing'), (104, 6, 'IT'), (105, 7, 'Finance');

--Inner Join Query Select Employee.emp_id ,Employee.name, Departments.department_name -- (these are extra) FROM Employee Inner Join Departments ON Employee.emp_id = Departments.emp_id;

<u>Left Join (All the values in the left table):</u>

Syntax:

Example of Left Join:

```
CREATE TABLE Customers (
   customer_id INT PRIMARY KEY,
   name VARCHAR(50) NOT NULL
);

INSERT INTO Customers (customer_id, name) VALUES
(1, 'Lolo'),
(2, 'Mpho'),
(3, 'Keketso');
```

```
-- Table 2: Orders
CREATE TABLE Orders (
   order_id INT PRIMARY KEY,
   customer_id INT,
   product VARCHAR(50) NOT NULL,
   FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)
);

INSERT INTO Orders (order_id, customer_id, product) VALUES
(101, 1, 'Laptop'),
(102, 2, 'Smartphone');

--LEFT JOIN
SELECT Customers.customer_id,Customers.name, Orders.order_id, Orders.product
FROM Customers
LEFT JOIN Orders
ON Customers.customer_id = Orders.customer_id;
```

RIGHT JOIN:

Syntax:

Example of Right Join:

```
CREATE TABLE Department (
   department_id INT PRIMARY KEY,
   department_name VARCHAR(50) NOT NULL
```

```
);

INSERT INTO Department (department_id, department_name) VALUES
(101, 'HR'),
(102, 'IT'),
(103, 'Marketing');

-- Table 2: Employees
CREATE TABLE Employees (
   employee_id INT PRIMARY KEY,
   name VARCHAR(50) NOT NULL,
   department_id INT,
   FOREIGN KEY (department_id) REFERENCES Department(department_id)
);

INSERT INTO Employees (employee_id, name, department_id) VALUES
(1, 'Kabo', 101),
(2, 'Emma', 102);

--RIGHT JOIN
SELECT Employees.name,Employees.department_id, Department.department_name
FROM Employees
RIGHT JOIN Department
ON Employees.department_id = Department.department_id;
```

Full Join/ Full outer join (Store records in a single table, whether they match or not):

Syntax for full join:

Example:

My Implementation:
-- Table 1: Sales CREATE TABLE SaleTable (sale_id INT PRIMARY KEY, customer_id INT, sale_amount DECIMAL(10,2) NOT NULL, sale_date DATE NOT NULL);

INSERT INTO SaleTable (sale_id, customer_id, sale_amount, sale_date) VALUES (1001, 201, 500.00, '2025-01-10'), (1002, 202, 1200.00, '2025-01-11'), (1003, 203, 300.00, '2025-01-12'), (1004, 205, 450.00, '2025-01-13');

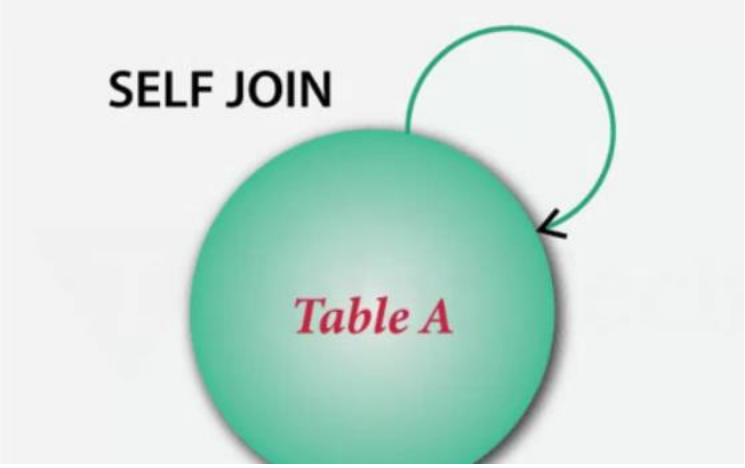
-- Table 2: Customer Feedback CREATE TABLE Customer_Feedback (feedback_id INT PRIMARY KEY, customer_id INT, feedback_score INT CHECK (feedback_score BETWEEN 1 AND 5), feedback_date DATE NOT NULL);

INSERT INTO Customer_Feedback (feedback_id, customer_id, feedback_score, feedback_date) VALUES (501, 202, 4, '2025-01-14'), (502, 203, 5, '2025-01-15'), (503, 204, 3, '2025-01-16');

--Full join SELECT * FROM SaleTable FULL JOIN Customer_Feedback ON SaleTable.customer_id = Customer_Feedback.customer_id;

Lecturer's implementation:
--QUERY TO PERFORM FULL JOIN SELECT SaleTable.customer_id, SaleTable.sale_amount, SaleTable.sale_date, Customer_Feedback.feedback_score, Customer_Feedback.feedback_date FROM SaleTable FULL JOIN Customer_Feedback ON SaleTable.customer_id = Customer_Feedback.customer_id;

SELF JOIN



Syntax of Self Join:

SYNTAX

```
SELECT
   A.column_name,
   B.column_name
FROM
   table_name AS A
JOIN
   table_name AS B
ON
   A.common_column = B.common_column;
```

Example:

We apparently use the Left join
-- Table: Employees CREATE TABLE Employees1 (emp_id INT PRIMARY KEY, name VARCHAR(50) NOT NULL, manager_id INT, CONSTRAINT FK_Employees_Manager FOREIGN KEY (manager_id) REFERENCES Employees1(emp_id)); --Matching the Employees with the manager INSERT INTO Employees1 (emp_id, name, manager_id) VALUES (1, 'Smanga', NULL), -- top-level manager (2, 'Thabile', 1), -- reports to Smanga (3, 'Ben', 1), -- reports to Smanga (4, 'Keletso', 2); -- reports to Thabile SELECT e.name AS employee, m.name AS manager FROM Employees1 AS e LEFT JOIN Employees1 AS m ON e.manager_id = m.emp_id;

CROSS JOIN:

Syntax:

Example:
Every product with Every Store

```
CREATE TABLE Products (
   product_id INT PRIMARY KEY,
   product_name VARCHAR(50) NOT NULL
);

INSERT INTO Products (product_id, product_name) VALUES
(1, 'Laptop'),
(2, 'Smartphone'),
(3, 'Headphones'),
(4, 'Smartwatch');

-- Table 2: Stores
CREATE TABLE Stores (
   store_id INT PRIMARY KEY,
   store_location VARCHAR(50) NOT NULL
);

INSERT INTO Stores (store_id, store_location) VALUES
(101, 'Kempton Park'),
(102, 'Midrand'),
(103, 'Thembisa');
```

--QUERY FOR PERFORMING CROSS JOIN

```
SELECT

p.product_name,

s.store_location

FROM

products p

CROSS JOIN

stores s;
```

CLASS EXERCISE IN OUR EMAILS:

5 Tables
--Customers CREATE TABLE Customers (customer_id INT PRIMARY KEY, customer_name VARCHAR(20), city VARCHAR(50)); INSERT INTO Customers (customer_id, customer_name, city) VALUES (101,'Scott' ,'Kempton Park'), (102,'Jazz' ,'Midrand'), (103,'Kayla' ,'Thembisa'), (104,'Xoli' ,'Benoni'), (105,'Sizwe' ,'Sandhurst'), (106,'Mike' ,'Illovo'), (107,'Poppy' ,'Centurion'), (108,'Sam' ,'Sandhurst'), (109,'Ben' ,'Illovo'), (110,'Zolile' ,'Benoni'), (111,'Ntombi' ,'Boksburg'), (112,'Sfiso' ,'Springs'), (113,'Themba' ,'Springs'), (114,'Junior' ,'Boksburg'), (115,'Sphesihle' ,'Thembisa'); --Branches CREATE TABLE Branches (branch_id INT PRIMARY KEY, branch_name VARCHAR(50), city VARCHAR(50)); INSERT INTO Branches VALUES (20,'Illovo Branch' ,'Illovo'), (40,'Sandhurst Branch' ,'Sandhurst'), (60,'Centurion Branch' ,'Centurion'), (80,'Midrand Branch' ,'Midrand'), (22,'Kempton Park Branch' ,'Kempton Park'), (24,'Thembisa Branch' ,'Thembisa'), (26,'Boksburg Branch' ,'Boksburg'), (28,'Benoni Branch' ,'Benoni'), (30,'Springs Branch' ,'Springs'); --Accounts CREATE TABLE Accounts (account_id INT PRIMARY KEY, customer_id INT, branch_id INT, account_type VARCHAR(20), balance DECIMAL(12,2), FOREIGN KEY (customer_id)

```
REFERENCES Customers(customer_id), FOREIGN KEY (branch_id) REFERENCES
Branches(branch_id) );

INSERT INTO Accounts VALUES (200,101,22,'Savings' ,3001.84), (201,102,80,'Cheque' ,198.34),
(202,103,24,'Cheque' ,0.77), (203,104,28,'Cheque' ,532.11), (204,105,40,'Savings' ,820.06),
(205,106,20,'Savings' ,52522.10), (206,107,60,'Cheque' ,74.25), (207,108,40,'Cheque' ,39.67),
(208,109,20,'Savings' ,12664.55), (209,110,28,'Savings' ,150.00),
(210,111,26,'Cheque' ,8400.33), (211,112,30,'Savings' ,13.97),
(212,113,30,'Savings' ,258639.40), (213,115,26,'Cheque' ,300.84),
(214,101,24,'Savings' ,300.84);

--Transactions CREATE TABLE Transactions ( transaction_id INT PRIMARY KEY, account_id INT,
transaction_date DATE, transaction_type VARCHAR(20), amount DECIMAL(12,2), FOREIGN KEY
(account_id) REFERENCES Accounts(account_id) );

INSERT INTO Transactions VALUES (1,200,'2025-11-11','Deposit' ,3000.00), (2,201,'2025-11-
12','Withdrawal' ,500.00), (3,202,'2025-11-19','Deposit' ,800.00), (4,203,'2025-11-
19','Deposit' ,20.00), (5,204,'2025-11-30','Withdrawal' ,300.00), (6,205,'2025-11-
06','Withdrawal' ,350.00), (7,206,'2025-11-01','Deposit' ,70.00), (8,207,'2025-11-
04','Withdrawal' ,200.00), (9,208,'2025-11-17','Deposit' ,100.00), (10,209,'2025-11-
12','Deposit' ,320.00), (11,210,'2025-11-10','Withdrawal' ,900.00), (12,211,'2025-11-
07','Withdrawal' ,40.00), (13,212,'2025-11-02','Deposit' ,10000.00), (14,213,'2025-11-
05','Deposit' ,10.00), (15,214,'2025-11-18','Withdrawal' ,210.00);

--Loans CREATE TABLE Loans ( loan_id INT PRIMARY KEY, customer_id INT, loan_amount
DECIMAL(12,2), interest_rate DECIMAL(5,2), FOREIGN KEY (customer_id) REFERENCES
Customers(customer_id) );

INSERT INTO Loans VALUES (001,101,null ,null), (002,102,null ,null), (003,103,18000,21),
(004,104,42000 ,28), (005,105,12200 ,22), (006,106,10000 ,19), (007,107,null ,null),
(008,108,29000 ,29), (009,109,25000 ,18.5), (0010,110,10000 ,25.50), (0011,111,16000 ,21.75),
(0012,112,68000 ,30), (0013,113,102000 ,16.5), (0014,114,12700 ,19.75),
(0015,115,19000 ,27.75);

--Part 2: Task 1 SELECT c.customer_id, c.customer_name, c.city, a.account_id,a.account_type,
a.balance FROM Customers c INNER JOIN Accounts a ON c.customer_id = a.customer_id;

--Task 2 SELECT Customers.customer_id,
Customers.customer_name,loan_amount,interest_rate FROM Customers LEFT JOIN Loans ON
Customers.customer_id = Loans.customer_id WHERE loan_amount IS NULL;

--Task 3 SELECT Branches.branch_id, Branches.branch_name, SUM(Accounts.balance) As
TotalAccountBalance FROM Branches LEFT JOIN Accounts ON Branches.branch_id =
Accounts.branch_id group by Branches.branch_id, Branches.branch_name;

--Task 4 SELECT A.city,B.customer_name FROM Customers As A INNER JOIN Customers As B
ON A.customer_id = B.customer_id group by A.city, B.customer_name;
```

```
/* Ronalds' version-- SELECT c1.customer_name AS customer_1, c2.customer_name AS customer_2, c1.city FROM Customers c1 INNER JOIN Customers c2 ON c1.city = c2.city AND c1.customer_id < c2.customer_id; */

--Task 5 SELECT Loans.customer_id,Loans.interest_rate, CASE WHEN Loans.interest_rate < 20 THEN 'LOW Risk Borrower' WHEN Loans.interest_rate >= 20 THEN 'HIGH Risk Borrower' ELSE 'NO Credit History, Offer Credit Product' END As Verdict FROM Loans;

--Task 6

--Task 7 SELECT Customers.customer_name, Accounts.account_id, COUNT(Transactions.transaction_id) AS transaction_count FROM Customers INNER JOIN Accounts ON Customers.customer_id = Accounts.customer_id INNER JOIN Transactions ON Accounts.account_id = Transactions.account_id GROUP BY Customers.customer_name, Accounts.account_id HAVING COUNT(Transactions.transaction_id) > 1;
```

TUESDAY(10/FEB/2026) - JOINS

RECAP OF JOINS

*How do we determine which joins to use?

-The results, the output that you want to see. Tells you which joins to use.

-There should be a common column between the tables.

Here's a Cheat Sheet of The JOINS

- INNER JOIN – Fetch data that is matching, from the right and left table, based on a specific condition (the condition being it should be the common column).

-Join, Inner Join, Left Join & Right Join are all similar.

-Cross Join and Self Join Are similar, they both use Aliases.

TRIGGERS

-Set of SQL statement with unique name stored in the computer memory

- A trigger is a special command

-To trigger events automatically (UPDATING, DELETING & INSERTING)

INSERT

MY EXAMPLE (**Insert**)1:

```
CREATE TABLE Employee(
Id INT PRIMARY KEY,
Name VARCHAR(45),
Salary INT,
Gender VARCHAR(12),
DepartmentId INT
);

INSERT INTO Employee VALUES
(1,'Phillips', 82000, 'Male', 3),
(2,'Jennifer', 52000, 'Female', 2),
(3,'Mbulelo', 25000, 'male', 1),
(4,'Kamo', 47000, 'Male', 2),
(5,'Priya', 46000, 'Female', 3);

CREATE TABLE Employee_Audit_Test
(
Id int IDENTITY,
Audit_Action text
```

```
)
SELECT * FROM Employee


CREATE TRIGGER trInsertEmployee
ON Employee
FOR INSERT
AS
BEGIN
   -- Instead of using a variable, we join the 'inserted' table
   -- directly into the Audit table. This handles 1 or 1,000 rows perfectly.
   INSERT INTO Employee_Audit_Test (Audit_Action)
   SELECT 'New employee with Id = ' + CAST(Id AS VARCHAR(10)) + ' is added at ' + CAST(GETDATE()
AS VARCHAR(22))
   FROM inserted;
END;


INSERT INTO Employee VALUES (6, 'Dumi', 60000, 'Male', 1);
SELECT * FROM Employee_Audit_Test;
```

https://www.geeksforgeeks.org/sql/sql-trigger-student-database/

Designed for multiple rows and not one?

1. **Here's the Syntax for insert without the Declare (For multiple rows at a time rather than one):**

```
CREATE TRIGGER tr_AfterInsert_TableName
ON TableName
FOR INSERT
AS
BEGIN
```

```
  INSERT INTO AuditTable (LogMessage)
  SELECT 'New record added with ID: ' + CAST(Id AS VARCHAR(10))
  FROM inserted; -- The "New" data
END;

-- To Test:
INSERT INTO TableName (Columns) VALUES (Values);
SELECT * FROM AuditTable;
```

SIR's Example:

DELETE

My Example (**Delete**)2:

```
--Delete Trigger
CREATE TRIGGER trDeleteEmployee2
ON Employee
FOR DELETE
AS
 BEGIN
 -- We select directly from the 'deleted' table to handle multiple rows
INSERT INTO Employee_Audit_Test (Audit_Action)
SELECT 'Employee with Id = ' + CAST(Id AS VARCHAR(10)) + ' was deleted at ' + CAST(GETDATE() AS VARCHAR(22))
```

FROM deleted; END;

SELECT * FROM Employee; SELECT * FROM Employee_Audit_Test;

DELETE FROM Employee WHERE Id = 3;

-- This should now show the deletion message for ID 3 SELECT * FROM Employee_Audit_Test;

2. Here's the Syntax for delete without the Declare (For multiple rows at a time rather than one):

```
CREATE TRIGGER tr_AfterDelete_TableName

ON TableName

FOR DELETE

AS

BEGIN

  INSERT INTO AuditTable (LogMessage)

  SELECT 'Record removed with ID: ' + CAST(Id AS VARCHAR(10))

  FROM deleted; -- The "Old" data that is now gone

END;


-- To Test:

DELETE FROM TableName WHERE Id = X;

SELECT * FROM AuditTable;
```

Sir's Example:

---WE ARE NOW CREATING A TRIGGER, STORING RECORDS OF EACH deletion ON EMP TBL. CREATE TRIGGER trDeleteEmployee ON Employee FOR DELETE AS BEGIN Declare @Id int SELECT

@Id = Id from deleted INSERT INTO Employee_Audit_Test VALUES ('An existing employee with Id = ' + CAST(@Id AS VARCHAR(10)) + ' is deleted at ' + CAST(Getdate() AS VARCHAR(22))) END

UPDATE

My Example (**UPDATE**)3:

```
CREATE TRIGGER tr_AfterUpdate_TableName

ON TableName

FOR UPDATE

AS

BEGIN

   INSERT INTO AuditTable (LogMessage)

   SELECT 'ID ' + CAST(i.Id AS VARCHAR(10)) +

        ' changed name from ' + d.Name + ' to ' + i.Name

   FROM inserted i

   JOIN deleted d ON i.Id = d.Id; -- Joining "New" (i) and "Old" (d)

END;


-- To Test:

UPDATE TableName SET Name = 'New Name' WHERE Id = X;

SELECT * FROM AuditTable;
```

3. Here's the Syntax for Update without the Declare (For multiple rows at a time rather than one):

CREATE TRIGGER tr_AfterUpdate_TableName

```
ON TableName
FOR UPDATE
AS
BEGIN
  INSERT INTO AuditTable (LogMessage)
  SELECT 'ID ' + CAST(i.Id AS VARCHAR(10)) +
      ' changed name from ' + d.Name + ' to ' + i.Name
  FROM inserted i
  JOIN deleted d ON i.Id = d.Id; -- Joining "New" (i) and "Old" (d)
END;

-- To Test:
UPDATE TableName SET Name = 'New Name' WHERE Id = X;
SELECT * FROM AuditTable;
```

Sir's Example:

USE OF SQL TRIGGERS

Types of SQL Triggers in SQL Server

TYPES OF DML Triggers

*After triggers are not going to run if the data insertion is not accurate or complete.

*These will run even if data insertion fails

WE DID AN EXAMPLE:

This is one code, all works well together
---1 Main table CREATE TABLE Employees (emp_id INT PRIMARY KEY, emp_name VARCHAR(50), department VARCHAR(50), salary DECIMAL(10,2)); --Audit / Log table CREATE TABLE Employee_Audit (audit_id INT IDENTITY(1,1) PRIMARY KEY, emp_id INT, action_type VARCHAR(20), old_salary DECIMAL(10,2), new_salary DECIMAL(10,2), action_date DATETIME DEFAULT GETDATE()); ---2 AFTER INSERT Trigger CREATE TRIGGER trg_AfterInsert_Employees ON Employees AFTER INSERT AS BEGIN INSERT INTO Employee_Audit (emp_id, action_type) SELECT emp_id, 'INSERT' FROM inserted; END; ----3 AFTER UPDATE Trigger CREATE TRIGGER trg_AfterUpdate_Employees ON Employees AFTER UPDATE AS BEGIN INSERT INTO Employee_Audit (emp_id, action_type, old_salary, new_salary) SELECT i.emp_id,--i is the alias and emp_id is the column name 'UPDATE',-- Just a string text d.salary AS old_salary, --alias d i.salary AS new_salary --alias i, salary is column name FROM inserted i -- INNER JOIN deleted d ON i.emp_id = d.emp_id; END; ---4 AFTER DELETE Trigger CREATE TRIGGER trg_AfterDelete_Employees ON Employees AFTER DELETE AS BEGIN INSERT INTO Employee_Audit (emp_id, action_type, old_salary) SELECT emp_id, 'DELETE', salary FROM deleted; END; ---Lets do a Quick Test INSERT INTO Employees VALUES (1, 'Steven', 'IT', 50000); SELECT * FROM Employees; UPDATE Employees SET salary = 55000 WHERE emp_id = 1; DELETE FROM Employees WHERE emp_id = 1; SELECT * FROM Employee_Audit;

WEDNESDAY(11/FEB/2026) - TRIGGERS PRACTICAL

SQL SERVER MANAGEMENT STUDIO

(SSMS) PRACTICAL

*This is an exercise that was sent on email, **Accounting_Triggers_Practical.sql**

THURSDAY(12/FEB/2026) - RECAP FOR HOW TO CREATE SQL TABLES

In preparation for tomorrow's presentation, look at Lwazi's Email to see another task.

PRIMARY KEY IDENTITY (2,3), this means it will start at 2 and increment by three units

DECIMAL (10,2), this means there are 10 digits accepted and 2 decimals after the coma.

DEFAULT 0, This can start at any number you want.

Here's how to drop a Foreign Key:
Alter table Transactions Drop Constraint FK TransactionsAccountsID;

RECAP OF OPERATORS:

SELECT * FROM Accounts1

WHERE AccountType = 'Savings'

AND Balance > 4000;

```
--LOGICAL OPERATOR
--savings accounts with balance above 4000
SELECT *FROM Accounts
WHERE AccountType = 'Savings'
AND Balance > 4000;    ---> (NOT EQUAL), OR(EITHER CONDITION), >=(GREATER OR EQUAL)
```

IF CHECK"

```
---CONDITIONAL STATEMENTS (IF ... ELSE)
DECLARE @Balance DECIMAL (10,2)
SET @Balance = 1000

IF @Balance >= 500
PRINT 'Withdrawal Allowed'
ELSE
PRINT 'Insufficient Funds'
```

IF checks the condition, and The CHECK looks for specific keywords that match (Matching value).

```
---CONDITIONAL STATEMENTS (IF ... ELSE)
DECLARE @Balance DECIMAL (10,2) --declare is creating variable, @balance is variable name
SET @Balance = 1000             --set is assigning value to variable @balance

IF @Balance >= 50000           ---if is checking condition, >= is the condition
PRINT 'Withdrawal Allowed'
ELSE                           --if true, print first message, if false print 2nd message
PRINT 'Insufficient Funds'
```

Three or more conditions

```
---HOW DO WE COMPARE 3 OR MORE CONDITIONS, CAN WE HAVE 3 OR MORE CONDITIONS?

--MULTIPLE CONDITIONS IN WHERE(AND, ON, IN, BETWEEN)
SELECT * FROM Accounts
WHERE AccountType = 'Savings'
AND Balance >5000
AND DateOpened>= '2026-01-01;
```

Mixing the AND and OR

```
SQLQuery1.sql - D...-DOME1RC\me (53))*

--MULTIPLE CONDITIONS IN WHERE(AND, ON, IN, BETWEEN)
SELECT * FROM Accounts
WHERE AccountType = 'Savings'
AND Balance >5000
AND DateOpened>= '2026-01-01';

SELECT * FROM Accounts
WHERE (AccountType = 'Savings' OR AccountType = 'Current')
AND Balance >7000
AND CustomerID = 2;
```

The code for the above.

---CONDITIONAL STATEMENTS (IF ... ELSE) DECLARE @Balance DECIMAL (10,2) --declare is creating variable, @balance is variable name SET @Balance = 1000 --set is assigning value to variable @balance

IF @Balance >= 50000 ---if is checking condition, >= is the condition PRINT 'Withdrawal Allowed' ELSE --if true, print first message, if false print 2nd message PRINT 'Insufficient Funds'

---HOW DO WE COMPARE 3 OR MORE CONDITIONS, CAN WE HAVE 3 OR MORE CONDITIONS?

--MULTIPLE CONDITIONS IN WHERE(AND, ON, IN, BETWEEN) SELECT * FROM Accounts WHERE AccountType = 'Savings' AND Balance >5000 AND DateOpened>= '2026-01-01';

SELECT * FROM Accounts WHERE (AccountType = 'Savings' OR AccountType = 'Current') AND Balance >7000 AND CustomerID = 2;

----project banking system. customers, accounts, transactions

--1st. create database CREATE DATABASE BankDB;

--2nd. create tables with primary & foreign key

CREATE TABLE Customers1(CustomerID INT PRIMARY KEY IDENTITY (1,1), FirstName VARCHAR (50) NOT NULL, LastName VARCHAR (50) NOT NULL, Email VARCHAR (100), Phone VARCHAR (15), DateCreated DATETIME DEFAULT GETDATE ());

INSERT INTO Customers1 (FirstName, LastName, Email, Phone) VALUES ('Thabo', 'Mokoena', 'thabo.mokoena@email.co.za', '0712345678'), ('Nomsa', 'Dlamini', 'nomsa.dlamini@email.co.za', '0723456789'), ('Sipho', 'Nkosi', 'sipho.nkosi@email.co.za', '0734567890'), ('Ayesha', 'Khan', 'ayesha.khan@email.co.za', '0745678901'), ('Johan', 'van der Merwe', 'johan.vdmerwe@email.co.za', '0767890123'), ('Lerato', 'Mabaso', 'lerato.mabaso@email.co.za', '0789012345'), ('Michael', 'Naidoo', 'michael.naidoo@email.co.za', '0790123456');

CREATE TABLE Accounts1(AccountsID INT PRIMARY KEY IDENTITY (1001,1), CustomerID INT NOT NULL, AccountType VARCHAR (20) CHECK (AccountType IN ('Savings', 'Current')), Balance DECIMAL (10,2) DEFAULT 0, DateOpened DATETIME DEFAULT GETDATE ()

CONSTRAINT FK_CustomerAccount FOREIGN KEY (CustomerID) REFERENCES Customers1(CustomerID)); INSERT INTO Accounts1 (CustomerID, AccountType, Balance) VALUES (1, 'Savings', 8500.00), (1, 'Current', 3200.00), (2, 'Savings', 12500.00), (3, 'Current', 2400.00), (4, 'Savings', 15750.00), (5, 'Current', 6800.00), (6, 'Savings', 9200.00), (7, 'Current', 4100.00);

CREATE TABLE Transactions1(TransactionsID INT PRIMARY KEY IDENTITY (1,1), AccountID INT NOT NULL, TransactionType VARCHAR (10) CHECK (TransactionType IN ('Deposits', 'Withdraw')), Amount DECIMAL (10,2) NOT NULL, TransactionDate DATETIME DEFAULT GETDATE ()

CONSTRAINT FK_TransactionsAccountsID FOREIGN KEY (AccountID) REFERENCES Accounts1(AccountsID)); INSERT INTO Transactions1 (AccountID, TransactionType, Amount, TransactionDate) VALUES (1001, 'Deposits', 2500.00, '2026-01-05 09:15:00'), (1002, 'Withdraw', 800.00, '2026-01-07 14:20:00'), (1003, 'Deposits', 1200.00, '2026-01-08 10:45:00'), (1004, 'Withdraw', 1500.00, '2026-01-09 16:30:00'), (1005, 'Deposits', 5000.00, '2026-01-10 11:00:00'), (1006, 'Withdraw', 2000.00, '2026-01-11 13:10:00'), (1007, 'Deposits', 3500.00, '2026-01-12 08:25:00'), (1008, 'Withdraw', 900.00, '2026-01-13 17:45:00');

SELECT * FROM Accounts1 WHERE AccountType = 'Savings' AND Balance > 4000;

DECLARE @Balance DECIMAL(10,2) = 8000; DECLARE @AccountType VARCHAR(20) = 'Savings'; DECLARE @CustomerAge INT = 25;

IF @Balance >= 5000 AND @AccountType = 'Savings' AND @CustomerAge >= 18 BEGIN PRINT 'Eligible for Premium Savings Account'; END ELSE BEGIN PRINT 'Not Eligible'; END

SELECT AccountsID, Balance, CASE WHEN Balance >= 15000 THEN 'Platinum' WHEN Balance >= 10000 THEN 'Gold' WHEN Balance >= 5000 THEN 'Silver' ELSE 'Standard' END AS AccountCategory FROM Accounts1;

With Inner Join they all share a common table:

---JOINS --INNER JOIN SELECT C.FirstName, C.LastName, A.AccountType, A.Balance FROM Customers1 C INNER JOIN Accounts A ON C.CustomerID = A.CustomerID;

Multiple Join Statement

--LEFT JOIN SELECT C.FirstName, A.AccountsID FROM Customers C ---C IS AN ALIAS LEFT JOIN Accounts A ---A IS AN ALIAS ON C.CustomerID = A.CustomerID;--JOINING CUSTOMERS TO ACCOUNTS

---MULTIPLE TABLE JOINS SELECT C.FirstName, A.AccountType, T.TransactionType, T.Amount FROM Customers1 C

INNER JOIN Accounts1 A ON C.CustomerID = A.CustomerID INNER JOIN Transactions1 T ON T.AccountID = A.AccountsID;

<u>TRIGGERS</u>

Here’s an INSERT Trigger:

FULL CODE WITH THE TRIGGER:

```
----project banking system. customers, accounts, transactions

--1st. create database CREATE DATABASE BankDB;

--2nd. create tables with primary & foreign key

CREATE TABLE Customers1( CustomerID INT PRIMARY KEY IDENTITY (1,1), FirstName VARCHAR (50) NOT NULL, LastName VARCHAR (50) NOT NULL, Email VARCHAR (100), Phone VARCHAR (15), DateCreated DATETIME DEFAULT GETDATE () );

INSERT INTO Customers1 (FirstName, LastName, Email, Phone) VALUES ('Thabo', 'Mokoena', 'thabo.mokoena@email.co.za', '0712345678'), ('Nomsa', 'Dlamini', 'nomsa.dlamini@email.co.za', '0723456789'), ('Sipho', 'Nkosi', 'sipho.nkosi@email.co.za', '0734567890'), ('Ayesha', 'Khan', 'ayesha.khan@email.co.za', '0745678901'), ('Johan', 'van der Merwe', 'johan.vdmerwe@email.co.za', '0767890123'), ('Lerato', 'Mabaso', 'lerato.mabaso@email.co.za', '0789012345'), ('Michael', 'Naidoo', 'michael.naidoo@email.co.za', '0790123456');

CREATE TABLE Accounts1( AccountsID INT PRIMARY KEY IDENTITY (1001,1), CustomerID INT NOT NULL, AccountType VARCHAR (20) CHECK (AccountType IN ('Savings', 'Current')), Balance DECIMAL (10,2) DEFAULT 0, DateOpened DATETIME DEFAULT GETDATE ()

CONSTRAINT FK_CustomerAccount FOREIGN KEY (CustomerID) REFERENCES Customers1(CustomerID) ); INSERT INTO Accounts1 (CustomerID, AccountType, Balance) VALUES (1, 'Savings', 8500.00), (1, 'Current', 3200.00), (2, 'Savings', 12500.00), (3, 'Current', 2400.00), (4, 'Savings', 15750.00), (5, 'Current', 6800.00), (6, 'Savings', 9200.00), (7, 'Current', 4100.00);

CREATE TABLE Transactions1( TransactionsID INT PRIMARY KEY IDENTITY (1,1), AccountID INT NOT NULL, TransactionType VARCHAR (10) CHECK (TransactionType IN ('Deposits', 'Withdraw')), Amount DECIMAL (10,2) NOT NULL, TransactionDate DATETIME DEFAULT GETDATE ()

CONSTRAINT FK_TransactionsAccountsID FOREIGN KEY (AccountID) REFERENCES Accounts1(AccountsID) ); INSERT INTO Transactions1 (AccountID, TransactionType, Amount, TransactionDate) VALUES (1001, 'Deposits', 2500.00, '2026-01-05 09:15:00'), (1002, 'Withdraw', 800.00, '2026-01-07 14:20:00'), (1003, 'Deposits', 1200.00, '2026-01-08 10:45:00'), (1004, 'Withdraw', 1500.00, '2026-01-09 16:30:00'), (1005, 'Deposits', 5000.00, '2026-01-10 11:00:00'), (1006, 'Withdraw', 2000.00, '2026-01-11 13:10:00'), (1007, 'Deposits', 3500.00, '2026-01-12 08:25:00'), (1008, 'Withdraw', 900.00, '2026-01-13 17:45:00');

SELECT * FROM Accounts1 WHERE AccountType = 'Savings' AND Balance > 4000;

DECLARE @Balance DECIMAL(10,2) = 8000; DECLARE @AccountType VARCHAR(20) = 'Savings'; DECLARE @CustomerAge INT = 25;
```

IF @Balance >= 5000 AND @AccountType = 'Savings' AND @CustomerAge >= 18 BEGIN PRINT 'Eligible for Premium Savings Account'; END ELSE BEGIN PRINT 'Not Eligible'; END

SELECT AccountsID, Balance, CASE WHEN Balance >= 15000 THEN 'Platinum' WHEN Balance >= 10000 THEN 'Gold' WHEN Balance >= 5000 THEN 'Silver' ELSE 'Standard' END AS AccountCategory FROM Accounts1;

---JOINS --INNER JOIN SELECT C.FirstName, C.LastName, A.AccountType, A.Balance FROM Customers1 C INNER JOIN Accounts A ON C.CustomerID = A.CustomerID;

--LEFT JOIN SELECT C.FirstName, A.AccountsID FROM Customers C ---C IS AN ALIAS LEFT JOIN Accounts A ---A IS AN ALIAS ON C.CustomerID = A.CustomerID;--JOINING CUSTOMERS TO ACCOUNTS

---MULTIPLE TABLE JOINS SELECT C.FirstName, A.AccountType, T.TransactionType, T.Amount FROM Customers1 C

INNER JOIN Accounts1 A ON C.CustomerID = A.CustomerID INNER JOIN Transactions1 T ON T.AccountID = A.AccountsID;

--TRIGGERS --WHEN A TRANSACTION IS INSERTED --IF DEPOSIT EXPECT ADDING TO BALANCE --IF WITHDRAW EXPECT SUBTRACTING FROM BALANCE

CREATE TRIGGER trg_UpdateBalance ---CREATED TRIGGER ON Transactions1 --TRIGGER ACTIVATING ON TRANSACTION TABLE AFTER INSERT ---RUN AFTER A NEW RECORD IS INSERTED AS BEGIN UPDATE Accounts1 ---WE ARE MODIFYING ACCOUNTS TABLE SET Balance = CASE --- WORKING LIFE IF INSIDE SQL WHEN i.TransactionType = 'Deposits' ---1ST WHEN, IF DEPOSITS TO ADD MONEY THEN Balance + i.Amount WHEN i.TransactionType = 'Withdraw' ---2ND WHEN IF WITHDRAW, SUBTRACT MONEY THEN Balance - i.Amount END FROM Accounts1 A INNER JOIN inserted i ---INSERTED IS SPECIAL TEMP TABLE ON A.AccountsID = i.AccountID; END; ---TEST TRIGGER INSERT INTO Transactions1 (AccountID, TransactionType, Amount) VALUES(1001, 'Deposits', 1000);

SELECT * FROM Accounts1 WHERE AccountsID = 1001;

THE INSERT TRIGGER ON ITS OWN:

--TRIGGERS --WHEN A TRANSACTION IS INSERTED --IF DEPOSIT EXPECT ADDING TO BALANCE --IF WITHDRAW EXPECT SUBTRACTING FROM BALANCE

CREATE TRIGGER trg_UpdateBalance ---CREATED TRIGGER ON Transactions1 --TRIGGER ACTIVATING ON TRANSACTION TABLE AFTER INSERT ---RUN AFTER A NEW RECORD IS INSERTED AS BEGIN UPDATE Accounts1 ---WE ARE MODIFYING ACCOUNTS TABLE SET Balance = CASE --- WORKING LIFE IF INSIDE SQL WHEN i.TransactionType = 'Deposits' ---1ST WHEN, IF DEPOSITS TO ADD MONEY THEN Balance + i.Amount WHEN i.TransactionType = 'Withdraw' ---2ND WHEN IF WITHDRAW, SUBTRACT MONEY THEN Balance - i.Amount END FROM Accounts1 A INNER JOIN inserted i ---INSERTED IS SPECIAL TEMP TABLE ON A.AccountsID = i.AccountID;

```
END; ---TEST TRIGGER INSERT INTO Transactions1 (AccountID, TransactionType, Amount)
VALUES(1001, 'Deposits', 1000);

SELECT * FROM Accounts1 WHERE AccountsID = 1001;
```

INTERMEDIATE

FRIDAY(13/FEB/2026) - JOINS

PRESENTATION

Ushanka, Fatima & Akhona

MONDAY(16/FEB/2026) - THE SETA PROGRAM **[WEEK3]**

Governance and Ethics

IT Governance Frameworks

COBIT
Governance & management of enterprise IT; processes, controls, and assurance.

PAIA
Promotion of Access to Information; transparency and records management.

King IV
Principles-based corporate governance emphasizing ethical leadership and IT oversight.

Implementation Focus

Scope & Obligations
Balancing privacy rights with access to information; ensuring governance, risk management, and audit readiness across the organization.
Roles & Responsibilities
Defining accountabilities for the Board, Executives, IT Managers, and Information Officers to ensure compliant decision-making.
Controls & Processes
Implementing robust policies, risk management frameworks, continuous monitoring, and reporting mechanisms.
Current Trends
Adapting governance for digital transformation, cloud services adoption, and evolving cybersecurity compliance requirements.

Sci-Pagorus

-**Email** of the same date, we had a class activity